

Proyecto de Título (Evaluación 2)



Integrantes:

Vicente Aravena

Ricardo Cuevas

Fabian Ortega

Profesor:

Cristian Marcelo Larrain Larrain

Ramo:

TALLER APLICADO DE PROGRAMACIÓN

Sección:

006V

2. ÍNDICE

3. Introducción

- 3.1. Descripción breve del proyecto
- 3.2. Objetivo general
- 3.3. Contexto del problema a resolver

4. Desarrollo Evaluación Parcial 1

5. Desarrollo Evaluación Parcial 2

a) Diagramas y documentación técnica

- a.1. Diagrama de arquitectura
- a.2. Diagramas de procesos / UML
- a.3. Modelo de datos
- a.4. Casos de uso
- a.5. Metodología de desarrollo utilizada
- a.6. Tecnologías consideradas para el proyecto

b) Configuración del ambiente de pruebas

- b.1. Configuración del servidor
- b.2. Instalación de lenguajes
- b.3. Bibliotecas utilizadas
- b.4. Frameworks y herramientas necesarias
- b.5. Explicación del ambiente de pruebas y cómo réplica producción

c) Evidencia técnica

- c.1. Procedimiento de respaldo de base de datos
- c.2. Configuración del servidor cloud/web
- c.3. Instalación de herramientas y librerías
- c.4. Capturas de pantalla / Evidencia de funcionamiento

d) Desarrollo del software

- d.1. Avance funcional del sistema/ Arq implementada / Funciones implementadas
- d.2. Buenas prácticas de programación
- d.3. Evidencia del código y funcionamiento

e) Plan y documentación de pruebas

- e.1. Casos de prueba / Validaciones realizadas
- e.2. Resultados obtenidos / Evidencia de pruebas ejecutadas

6. Conclusiones

- 6.1. Estado actual del proyecto
- 6.2. Dificultades encontradas
- 6.3. Próximos pasos

7. Lecciones aprendidas

3. INTRODUCCIÓN

3.1. Descripción breve del proyecto

WorkSí es una plataforma de reclutamiento laboral orientada a optimizar el proceso de búsqueda y selección de empleo mediante el uso de inteligencia artificial. El proyecto contempla una aplicación móvil para postulantes y una plataforma web para administradores y reclutadores de empresas, permitiendo gestionar ofertas laborales, postulaciones y procesos de selección de manera centralizada.

El sistema utiliza un modelo de matching basado en inteligencia artificial, el cual analiza el contenido de los currículums vitae y lo compara con la información relevante de las ofertas laborales para generar un porcentaje de compatibilidad entre candidatos y puestos de trabajo. Para ello, el proyecto implementa un servicio desacoplado en FastAPI encargado del procesamiento de texto, embeddings y cálculo de similitud semántica.

La solución está desarrollada con una arquitectura compuesta por un backend monolítico en Spring Boot, una base de datos MySQL, una aplicación web construida con React y Vite, y una aplicación móvil desarrollada en Kotlin con Jetpack Compose. Además, el sistema considera medidas orientadas a reducir sesgos en el proceso de matching, aplicando técnicas de limpieza y anonimización de datos sensibles antes del análisis realizado por la inteligencia artificial.

3.2. Objetivo general

Desarrollar una plataforma de reclutamiento laboral denominada WorkSí, compuesta por una aplicación móvil para postulantes y una plataforma web para administradores y reclutadores, que permita gestionar ofertas de empleo y postulaciones mediante un sistema de matching basado en inteligencia artificial, con el propósito de optimizar los procesos de selección de personal, facilitar la búsqueda de empleo y reducir posibles sesgos en la evaluación inicial de candidatos.

3.3. Contexto del problema a resolver

Actualmente, los procesos de búsqueda y selección de personal presentan diversas dificultades tanto para los postulantes como para las empresas. Por una parte, los candidatos suelen enfrentarse a procesos extensos y poco eficientes para encontrar ofertas laborales acordes a sus habilidades e intereses. Por otra parte, las empresas y reclutadores deben revisar una gran cantidad de postulaciones de forma manual, lo que incrementa el tiempo y la dificultad para identificar perfiles adecuados.

Además, los métodos tradicionales de reclutamiento pueden verse influenciados por sesgos asociados a información personal o antecedentes que no necesariamente reflejan las competencias reales de un postulante. Esto puede afectar la objetividad en las primeras etapas de evaluación y disminuir la eficiencia del proceso de selección.

Frente a esta problemática, WorkSí propone una solución tecnológica que integra inteligencia artificial para analizar y comparar automáticamente la información relevante de los currículums y las ofertas laborales, generando porcentajes de compatibilidad que apoyan la toma de decisiones. De esta manera, se busca agilizar el proceso de reclutamiento, facilitar la experiencia de los usuarios y contribuir a evaluaciones iniciales más objetivas y enfocadas en las capacidades profesionales de cada candidato.

4. DESARROLLO EVALUACIÓN PARCIAL 1

Informe primera evaluación

<https://docs.google.com/document/d/147n-serMIKfrfES1o7rEpq3SGnF75vIldqJEq3FjZ1Y/edit?usp=sharing>

5. DESARROLLO EVALUACIÓN PARCIAL 2

a) Diagramas y documentación técnica

a.1. Diagrama de arquitectura

En este proyecto, la arquitectura se organiza mediante una aplicación móvil para candidatos, una plataforma web para administradores y reclutadores, un backend monolítico en Spring Boot, una base de datos MySQL, un servicio de inteligencia artificial en FastAPI y almacenamiento de archivos en filesystem.

La arquitectura propuesta separa las responsabilidades del sistema en distintas capas. La aplicación móvil, desarrollada en Kotlin con Jetpack Compose, permite a los candidatos registrarse, subir su CV, visualizar ofertas, realizar swipe y postular. Por otro lado, la plataforma web, desarrollada con React y Vite, permite al administrador registrar empresas y reclutadores, mientras que los reclutadores pueden crear ofertas, revisar postulantes y marcar postulaciones como vistas.

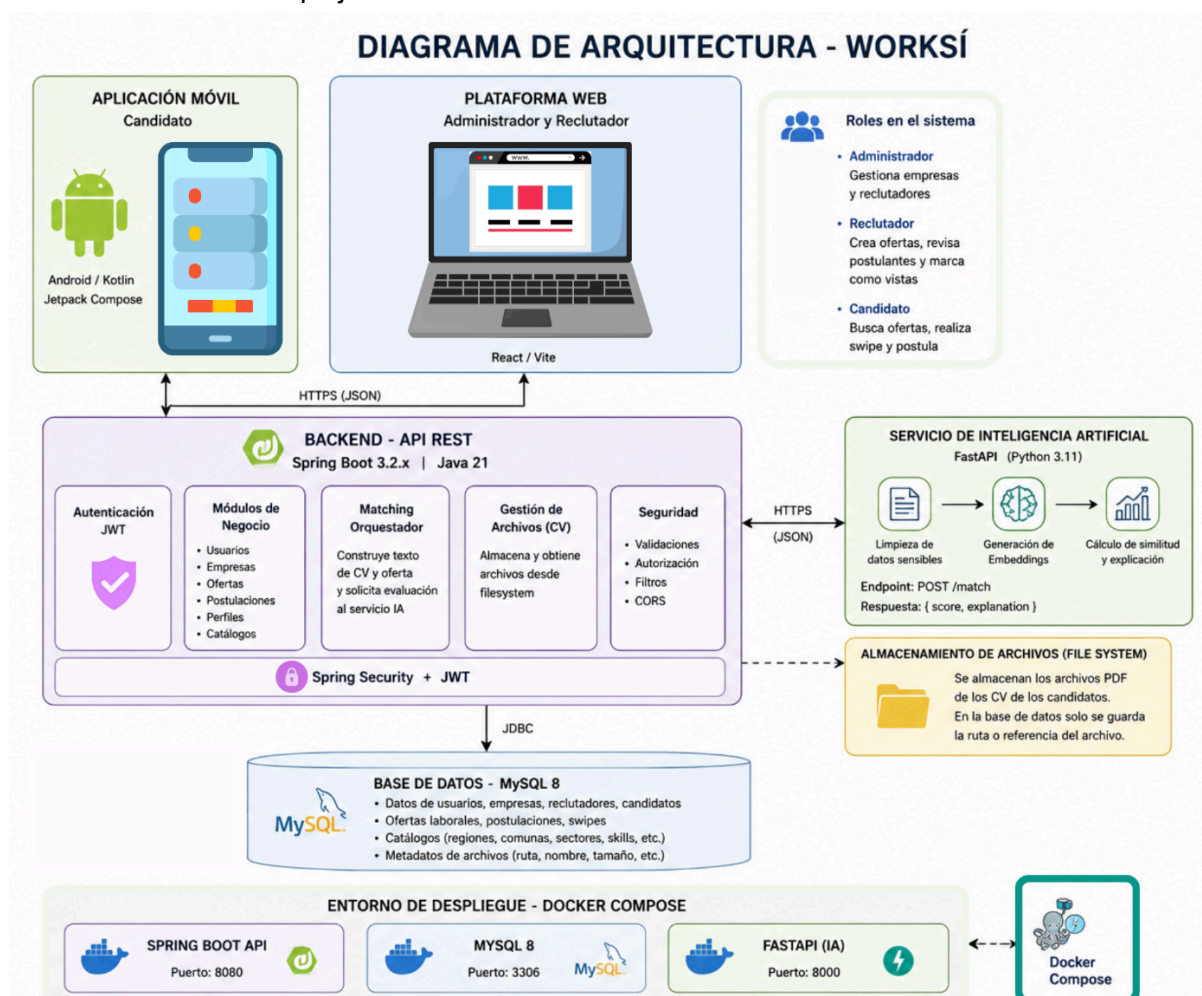
El backend central del sistema está desarrollado en Java 21 con Spring Boot 3.2.x. Este componente expone la API REST utilizada por la aplicación móvil y la plataforma web, gestiona la autenticación mediante JWT, aplica las reglas de negocio, maneja las

operaciones sobre la base de datos y coordina la comunicación con el servicio de inteligencia artificial.

La base de datos utilizada es MySQL 8, administrada mediante migraciones Flyway. En ella se almacenan usuarios, empresas, perfiles de reclutadores, candidatos, ofertas, postulaciones, catálogos y metadatos de archivos. Los currículums no se almacenan como binarios dentro de la base de datos, sino como archivos en el filesystem del servidor, manteniendo en MySQL únicamente la referencia o ruta correspondiente.

El servicio de inteligencia artificial está desacoplado del backend principal y se implementa con Python 3.11 y FastAPI. Spring Boot se comunica con este servicio mediante HTTP, enviando el texto extraído del CV y el texto relevante de la oferta al endpoint /match. Este servicio aplica limpieza de datos sensibles, genera embeddings, calcula similitud semántica y devuelve un score junto con una explicación simple del resultado.

Para el despliegue del ambiente se considera Docker Compose, integrando al menos los servicios de MySQL, Spring Boot y FastAPI. La aplicación Android corre de manera externa a Docker, consumiendo la misma API REST que la plataforma web. Esta arquitectura busca mantener una solución simple, funcional y adecuada al alcance MVP del proyecto, evitando microservicios adicionales, colas, Kubernetes o mecanismos de infraestructura innecesariamente complejos.



a.2. Diagramas de procesos/UML

Para WorkSí se consideran como procesos principales:

1. Registro de candidato

El candidato completa sus datos, sube su CV, selecciona skills y preferencias, acepta el consentimiento de uso de datos y finalmente se crea su cuenta en el sistema.

2. Publicación de oferta laboral

El reclutador inicia sesión, completa los datos de la oferta, selecciona modalidad, carga horaria y skills, y publica la oferta para que esté disponible para los candidatos.

3. Proceso de matching con IA

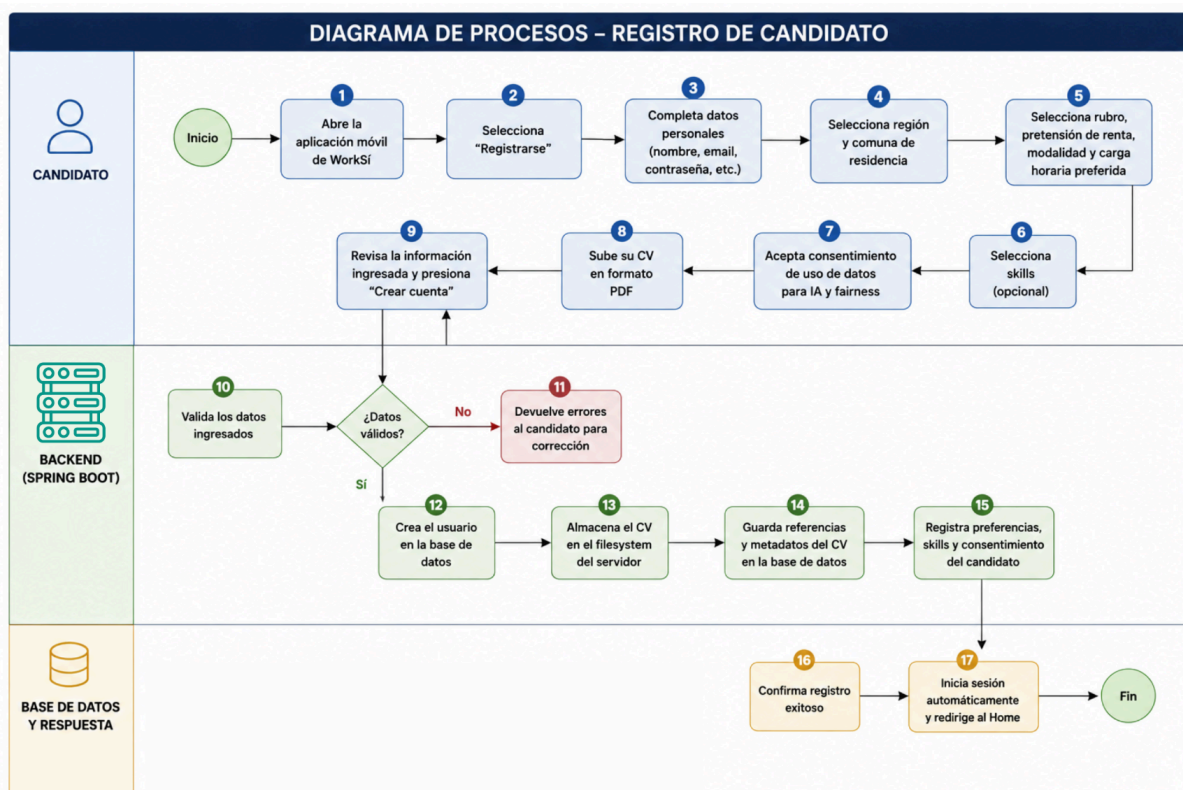
El backend extrae el texto del CV, construye el texto relevante de la oferta, envía ambos al servicio de IA, se aplica limpieza de datos sensibles, se calcula la similitud y se devuelve un porcentaje de compatibilidad.

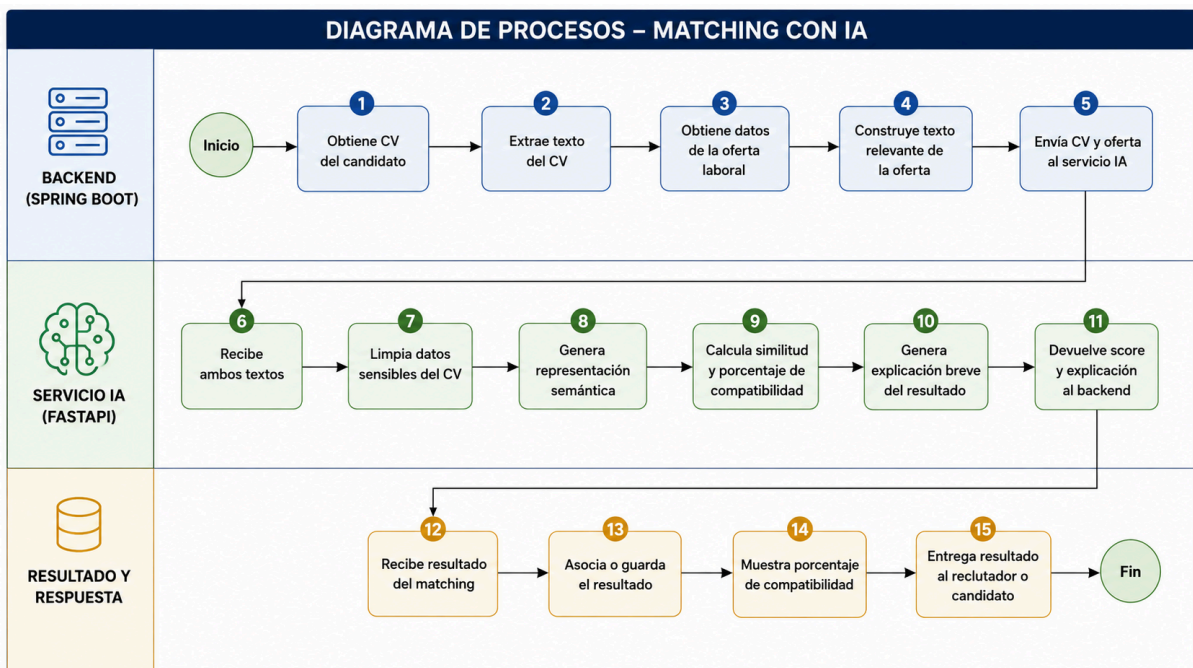
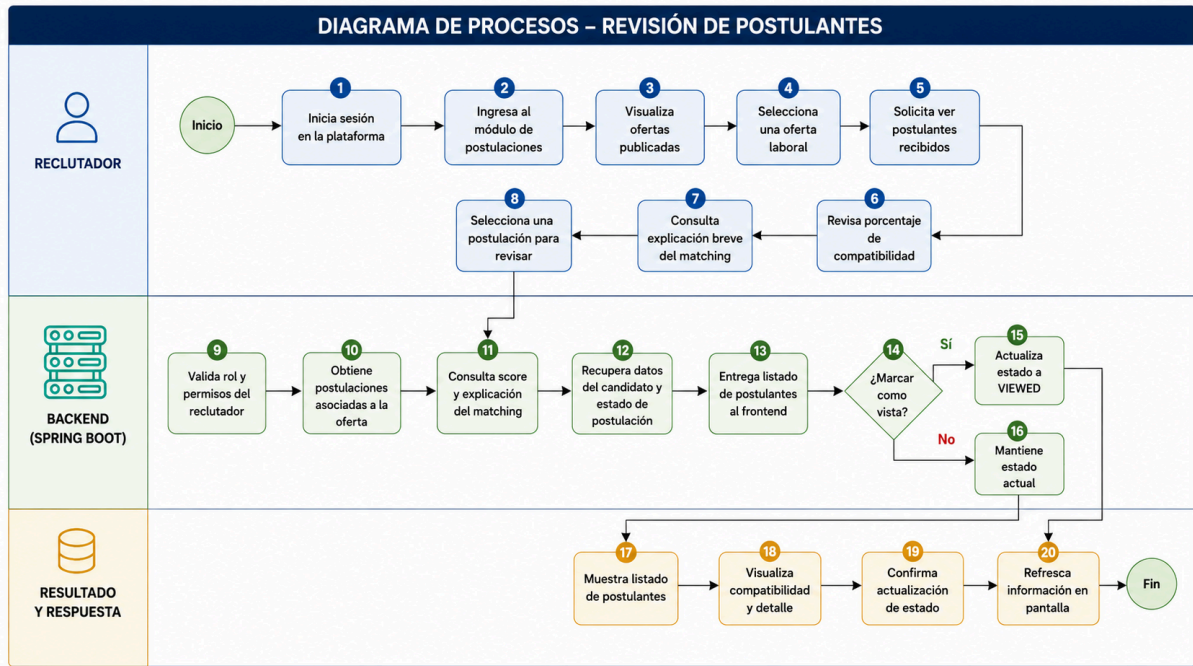
4. Postulación del candidato

El candidato visualiza ofertas, revisa el porcentaje de compatibilidad, realiza swipe y confirma su postulación a una oferta laboral.

5. Revisión de postulantes por reclutador

El reclutador visualiza las postulaciones recibidas, revisa el porcentaje de compatibilidad de los candidatos y puede marcar una postulación como vista.





Proceso de matching con IA

El backend extrae el texto del CV, construye el texto relevante de la oferta, envía ambos al servicio de IA, se aplica limpieza de datos sensibles, se calcula la similitud y se devuelve un porcentaje de compatibilidad.

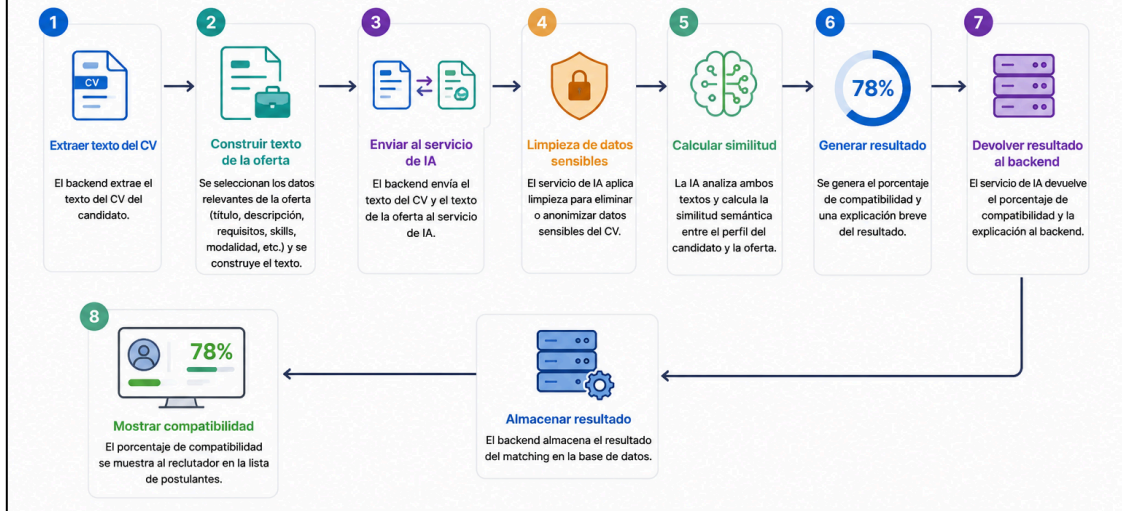
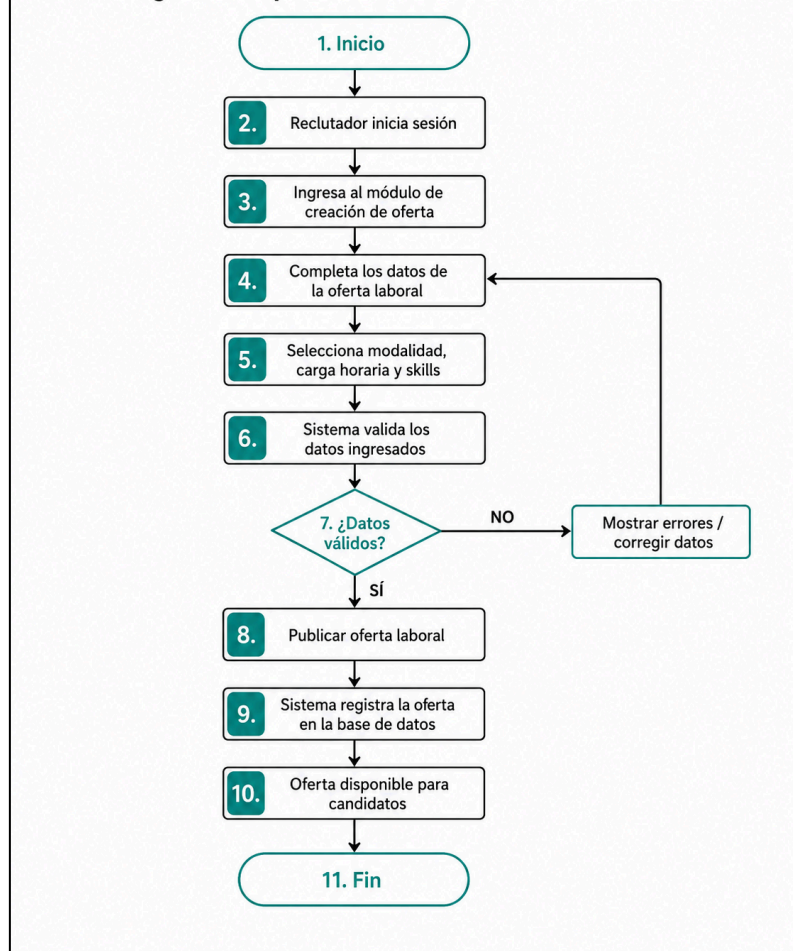
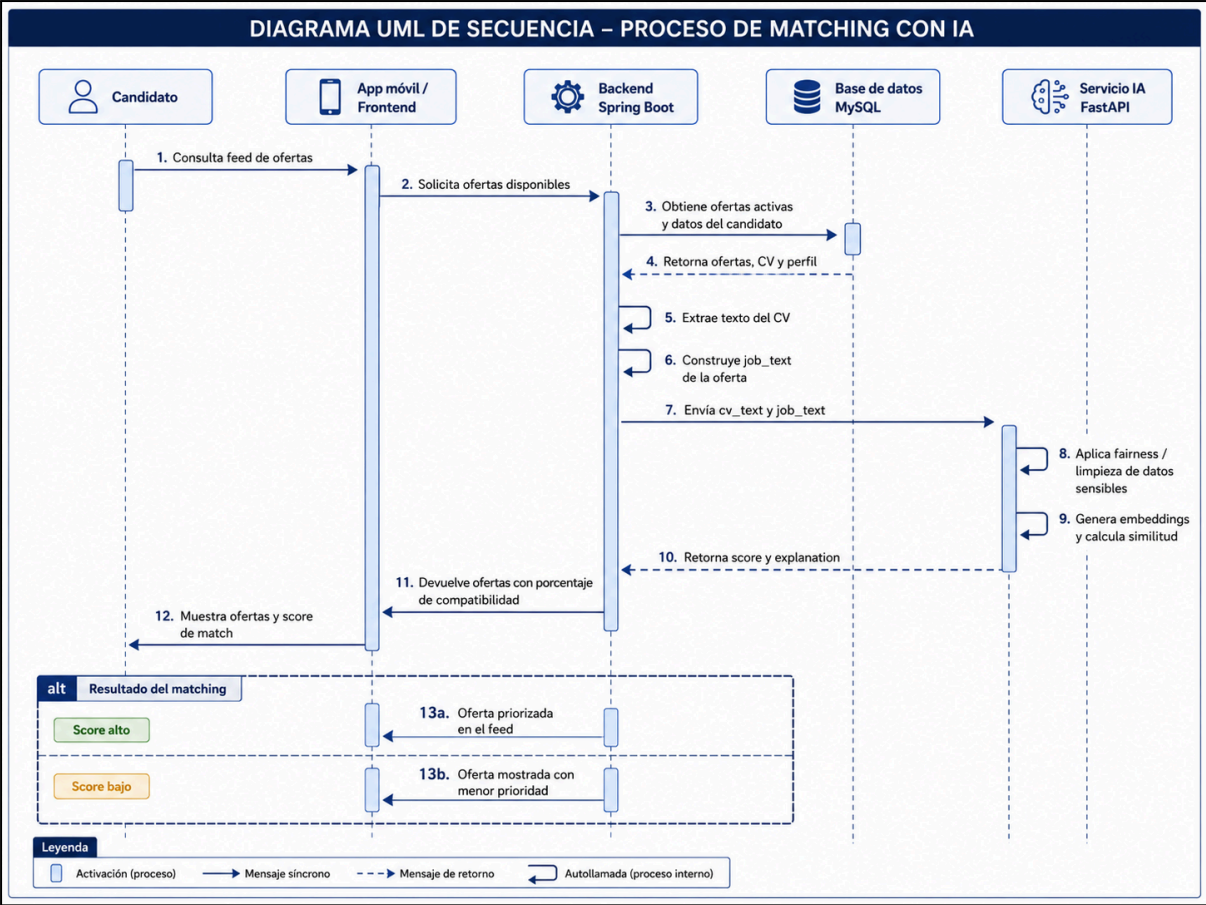


Diagrama de proceso: Publicación de oferta laboral





a.3. Modelo de datos

El modelo de datos de WorkSí se basa en una base de datos relacional MySQL 8, administrada mediante migraciones versionadas con Flyway. Su estructura está orientada a representar los usuarios del sistema, las empresas, los reclutadores, los candidatos, las ofertas laborales, las postulaciones y los catálogos necesarios para el funcionamiento de la plataforma. Las tablas principales consideradas en el modelo son:

users: almacena las cuentas con acceso al sistema. Los roles definidos son CANDIDATE, ADMIN y RECRUITER. En esta tabla se guarda el correo, el hash de la contraseña y la información necesaria para autenticación.

companies: almacena la ficha de empresa. La empresa no posee una cuenta de usuario propia, sino que funciona como entidad de negocio vinculada a reclutadores. Incluye datos como nombre comercial, razón social, correo corporativo, teléfono, dirección, región, comuna y sector.

recruiter_profiles: relaciona a los usuarios con rol RECRUITER con una empresa específica mediante `company_id`. Esto permite que cada reclutador pueda gestionar las ofertas asociadas a su empresa.

candidate_profiles: almacena la información del perfil del candidato, como datos personales, comuna, región, rubro, pretensión de renta, preferencias de modalidad, carga horaria y consentimiento de uso de datos.

candidate_cvs: registra los metadatos de los currículums subidos por los candidatos. Los archivos PDF se almacenan en el filesystem del servidor, mientras que en la base de datos solo se guarda la referencia, ruta o URL del archivo, evitando guardar el binario directamente en MySQL.

jobs: almacena las ofertas laborales creadas por reclutadores. Cada oferta se asocia a una empresa mediante `company_id` y contiene datos como título, descripción, ubicación, sueldo ofrecido, años de experiencia requeridos, modalidad, carga horaria, estado e imagen opcional.

applications: registra las postulaciones realizadas por los candidatos a las ofertas laborales. Sus estados principales son APPLIED, VIEWED y CANCELLED, aunque en el MVP se consideran visibles principalmente APPLIED y VIEWED.

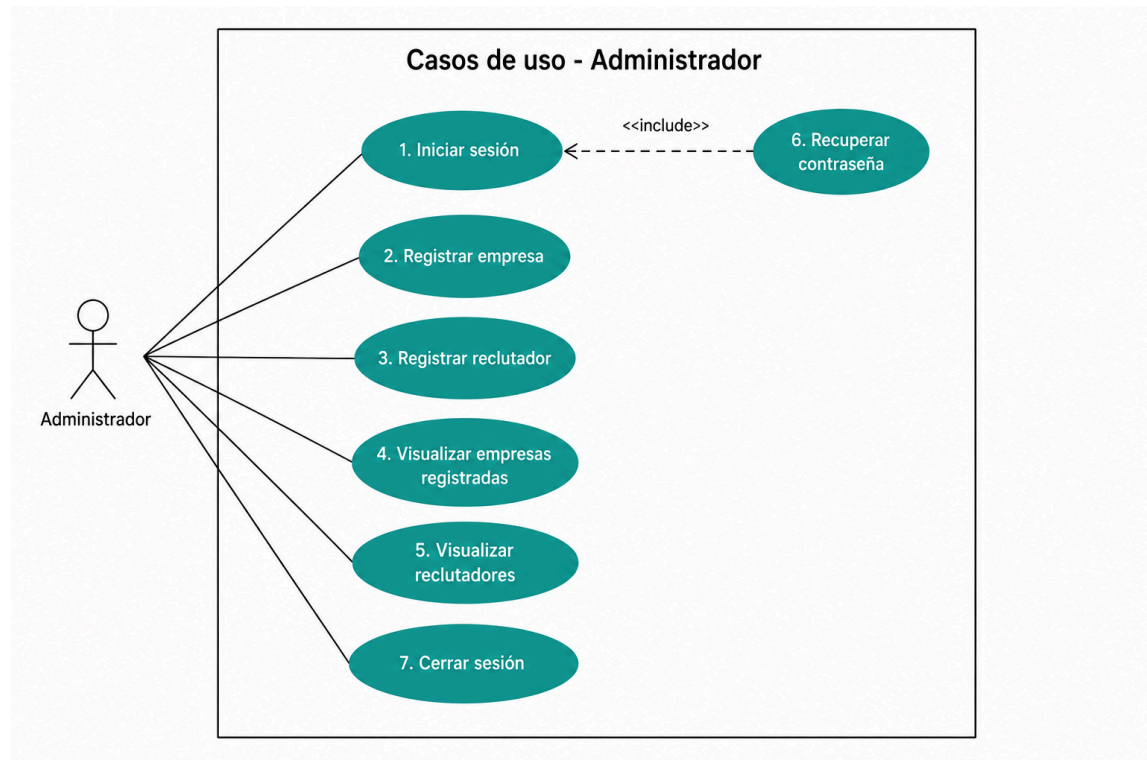
swipes: permite registrar las acciones realizadas por el candidato sobre las ofertas, como pasar una oferta o postular, evitando que se repitan ofertas ya vistas.

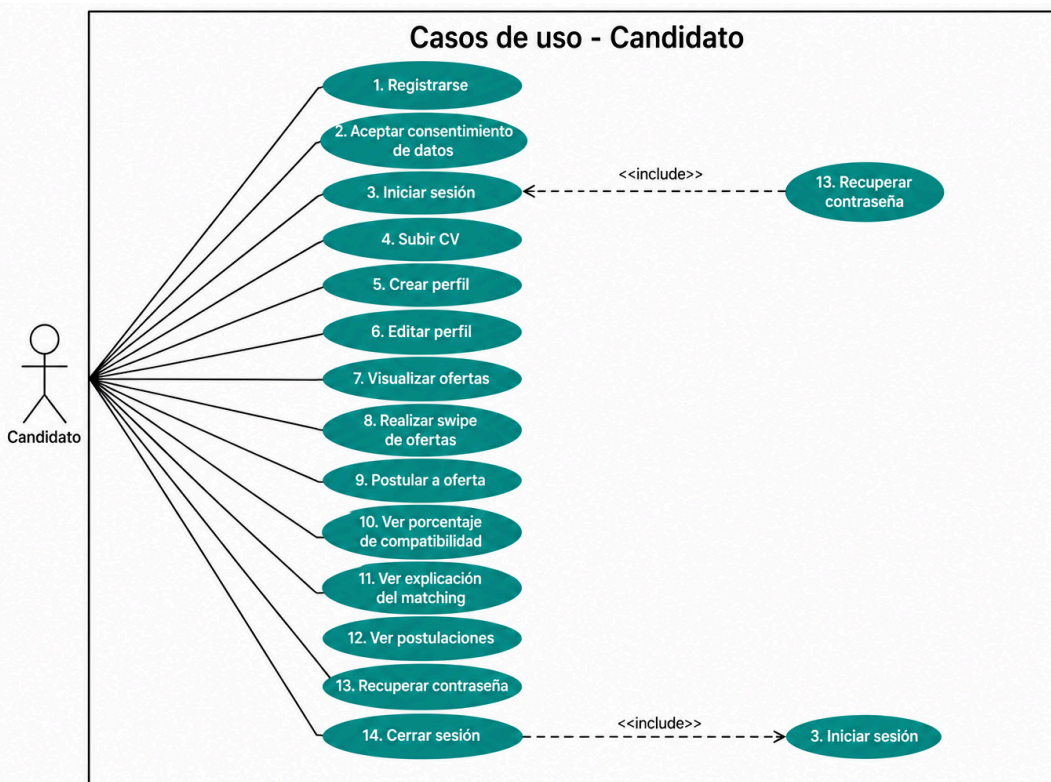
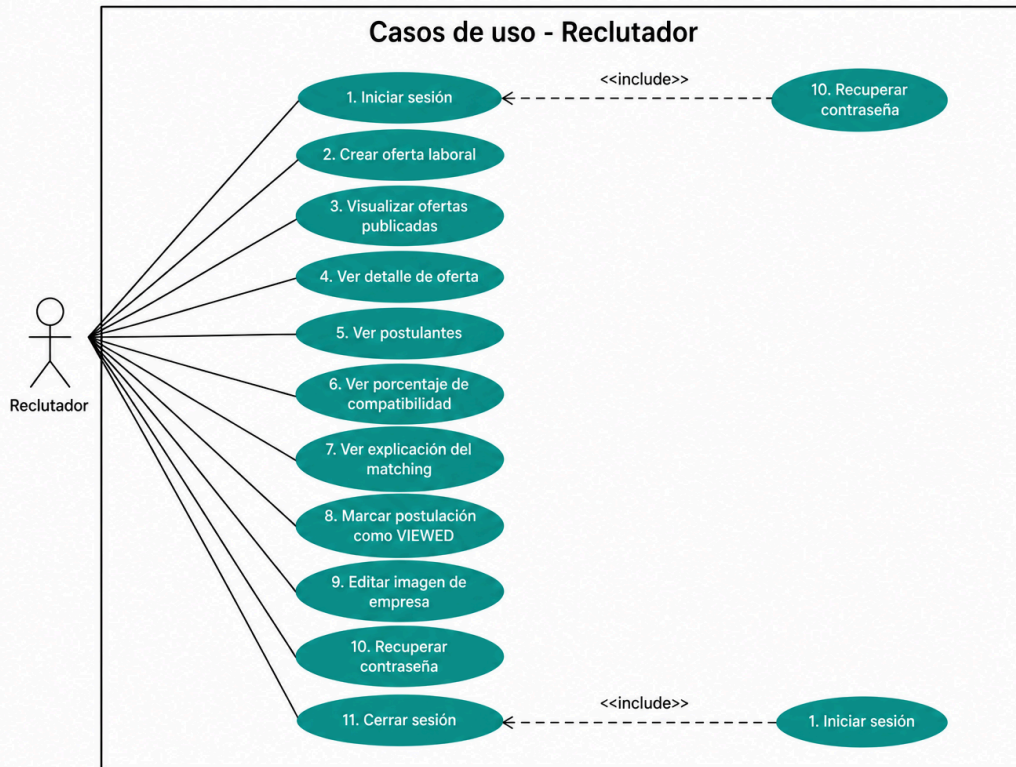
catalog tables: incluyen regiones, comunas, sectores y skills. Estas tablas permiten alimentar los formularios de registro, perfiles y creación de ofertas laborales.

Además, el sistema no utiliza una tabla matches, ya que el cálculo de compatibilidad se realiza dinámicamente mediante el servicio de inteligencia artificial. El backend construye el texto de la oferta y lo compara con el texto extraído del CV, mientras que el resultado del matching se obtiene en tiempo de ejecución.

a.4. Casos de uso

Los siguientes diagramas de casos de uso representan las principales funcionalidades del sistema WorkSí y la interacción de cada tipo de usuario con la plataforma. En ellos se modelan las acciones que pueden realizar los distintos actores del proyecto, considerando los procesos definidos para la aplicación móvil de candidatos y la plataforma web de administración y reclutamiento.





a.5. Metodología de desarrollo utilizada

Para el desarrollo del proyecto WorkSí se utilizó una metodología ágil basada en Scrum, organizada mediante un total de 11 sprints semanales. Esta metodología permitió dividir el proyecto en etapas incrementales, facilitando la planificación, el seguimiento del avance y la integración progresiva de las distintas funcionalidades del sistema.

Cada sprint fue diseñado considerando prioridades funcionales y dependencias técnicas, permitiendo desarrollar de manera ordenada aspectos como autenticación, gestión de usuarios, procesamiento de currículums, creación de ofertas laborales, matching con inteligencia artificial y visualización de postulaciones. Además, la metodología utilizada favoreció la distribución de tareas entre los integrantes del equipo y la validación continua de los avances obtenidos.

El enfoque de desarrollo se centró en la construcción de un Producto Mínimo Viable (MVP), priorizando las funcionalidades esenciales del sistema y evitando la sobreingeniería. Esto permitió enfocar los esfuerzos en los componentes más importantes del proyecto, como el sistema de matching basado en inteligencia artificial, el flujo de postulaciones y la gestión de ofertas laborales.

Asimismo, el proyecto fue organizado siguiendo una planificación incremental, donde cada sprint incorporó nuevas funcionalidades y mejoras sobre la base previamente desarrollada, permitiendo mantener un flujo constante de integración y pruebas funcionales durante el proceso de desarrollo.

a.6. Tecnologías consideradas para el proyecto

El proyecto WorkSí fue desarrollado utilizando un conjunto de tecnologías orientadas a la construcción de una arquitectura moderna, escalable y funcional, separando las responsabilidades entre backend, frontend web, aplicación móvil, base de datos e inteligencia artificial.

Backend

Para el desarrollo del backend se utilizó Java 21 junto con Spring Boot 3.2.x, permitiendo implementar una arquitectura monolítica basada en servicios REST. Además, se integraron las siguientes herramientas y tecnologías:

- Spring Security para autenticación mediante JWT.
- JPA/Hibernate para el manejo de persistencia de datos.
- Flyway Community para el control y versionado de migraciones SQL.
- Apache Tika para la extracción y procesamiento de texto desde archivos PDF de currículum.

Base de datos

Se utilizó MySQL 8 como sistema gestor de bases de datos relacional, debido a su estabilidad, facilidad de integración con Spring Boot y capacidad para manejar relaciones entre entidades del sistema.

Aplicación móvil

La aplicación móvil para postulantes fue desarrollada en Kotlin utilizando Android Studio y Jetpack Compose, implementando una interfaz basada en componentes modernos y manejo de estados mediante ViewModel.

Plataforma web

La plataforma web destinada a administradores y reclutadores fue desarrollada utilizando:

- React
- Vite
- React Router

Este conjunto de tecnologías permitió construir una interfaz web dinámica y orientada a componentes reutilizables.

Inteligencia Artificial

El sistema de matching fue implementado mediante un servicio desacoplado desarrollado en Python 3.11 utilizando FastAPI. Para el procesamiento de lenguaje natural y cálculo de similitud se consideraron las siguientes herramientas:

- sentence-transformers
- spaCy
- scikit-learn

Además, se utilizó el modelo de embeddings all-MiniLM-L6-v2 para calcular similitud semántica entre currículums y ofertas laborales.

Infraestructura y contenedores

Para facilitar el despliegue y la integración entre servicios se utilizó:

- Docker
- Docker Compose

La infraestructura considera contenedores separados para:

- Backend Spring Boot
- Servicio de IA FastAPI
- Base de datos MySQL

Asimismo, se implementó almacenamiento de archivos CV en filesystem, manteniendo en la base de datos únicamente la referencia o ruta del archivo.

b) Configuración del ambiente de pruebas

b.1. Configuración del servidor

Para el proyecto WorkSí se considera una configuración de servidor basada en un entorno contenerizado mediante Docker y Docker Compose. Esta configuración permite ejecutar de forma ordenada los servicios principales del sistema, facilitando la integración entre el backend, la base de datos y el servicio de inteligencia artificial.

El servidor contempla los siguientes componentes principales:

- **Backend Spring Boot:** encargado de exponer la API REST del sistema, gestionar la autenticación mediante JWT, aplicar las reglas de negocio y comunicarse con la base de datos y el servicio de IA.
- **Base de datos MySQL 8:** utilizada para almacenar usuarios, empresas, reclutadores, candidatos, ofertas, postulaciones, catálogos y metadatos de archivos.
- **Servicio de inteligencia artificial FastAPI:** encargado de recibir el texto del CV y de la oferta laboral, aplicar limpieza de datos sensibles, generar embeddings y calcular el porcentaje de compatibilidad.
- **Filesystem del servidor:** utilizado para almacenar los archivos PDF de los currículos, guardando en la base de datos únicamente la ruta o referencia del archivo.

La comunicación entre los componentes se realiza principalmente mediante HTTP y JDBC. La aplicación web y la aplicación móvil consumen la API REST del backend Spring Boot, mientras que el backend se comunica internamente con FastAPI a través del endpoint /match. La conexión con MySQL se realiza mediante JDBC.

En el ambiente de pruebas, Docker Compose permite levantar los servicios necesarios de manera conjunta, definiendo puertos, variables de entorno, credenciales de base de datos y volúmenes persistentes. Esto permite simular un entorno similar al de producción, manteniendo separados los servicios y asegurando que el backend, la base de datos y el servicio de IA puedan ejecutarse de forma integrada.

b.2. Instalación de lenguajes

Para el backend fue necesario instalar **Java 21**, debido a Spring Boot. Además, se configuraron herramientas como Maven y las variables de entorno correspondientes para compilar y ejecutar la API REST.

En el desarrollo del frontend web, el equipo debió instalar **Node.js** y **npm**, necesarios para ejecutar el proyecto construido con React y Vite. Gracias a esto fue posible instalar dependencias mediante comandos como npm install y levantar el entorno de desarrollo utilizando npm run dev.

Para la aplicación móvil se instaló **Android Studio**, junto con el SDK de Android y el lenguaje **Kotlin**, permitiendo desarrollar y ejecutar la aplicación móvil mediante Jetpack Compose y emuladores Android.

En el módulo de inteligencia artificial fue necesario instalar **Python 3.11** y posteriormente configurar las librerías utilizadas para el procesamiento de texto y cálculo de compatibilidad, tales como FastAPI, sentence-transformers, spaCy y scikit-learn.

Además, el equipo instaló **Docker** y **Docker Compose** para facilitar la ejecución integrada de los distintos servicios del sistema, y finalmente, se instaló **MySQL 8** como sistema gestor de bases de datos y se configuró Flyway para administrar las migraciones SQL versionadas utilizadas en el proyecto.

b.3. Bibliotecas utilizadas

En el backend desarrollado con Spring Boot se utilizaron principalmente las siguientes bibliotecas:

- **Spring Security:** utilizada para implementar autenticación y autorización mediante JWT.
- **Hibernate / JPA:** utilizada para el mapeo objeto-relacional y manejo de persistencia con MySQL.
- **Flyway:** utilizada para gestionar migraciones SQL versionadas.
- **Apache Tika:** utilizada para extraer texto desde archivos PDF de currículum.
- **JWT libraries:** utilizadas para generar y validar tokens de autenticación.

En el frontend web desarrollado con React y Vite se utilizaron bibliotecas relacionadas con el manejo de rutas, componentes y consumo de API REST, entre ellas:

- **React Router:** utilizada para navegación entre vistas.
- Dependencias de Vite y React necesarias para la construcción del proyecto web.

En la aplicación móvil Android se utilizaron bibliotecas orientadas al desarrollo moderno de interfaces y consumo de servicios:

- **Jetpack Compose:** utilizada para construir la interfaz gráfica de la aplicación.
- **ViewModel:** utilizada para manejo de estados y lógica de presentación.
- **Retrofit:** utilizada para consumir la API REST del backend.
- **OkHttp Interceptors:** utilizados para incorporar automáticamente el token JWT en las solicitudes.

Para el servicio de inteligencia artificial desarrollado en Python se utilizaron las siguientes bibliotecas:

- **FastAPI:** utilizada para construir el servicio REST de IA.
- **sentence-transformers:** utilizada para generar embeddings semánticos.
- **spaCy:** utilizada para reconocimiento de entidades y apoyo en fairness.
- **scikit-learn:** utilizada para calcular similitud mediante cosine similarity.

Estas bibliotecas permitieron implementar de manera eficiente las distintas funcionalidades del sistema, manteniendo una arquitectura coherente con el alcance MVP definido para el proyecto.

b.4. Frameworks y herramientas necesarias

En el backend se utilizó **Spring Boot 3.2.x** como framework principal para el desarrollo de la API REST, permitiendo estructurar el proyecto mediante controladores, servicios y repositorios. Además, se utilizó **Spring Security** para implementar autenticación y autorización mediante JWT.

En la plataforma web se trabajó con **React** como framework para la construcción de interfaces dinámicas y reutilizables, mientras que **Vite** fue utilizado como herramienta de desarrollo y compilación del proyecto frontend.

Para la aplicación móvil se utilizó **Android Studio** como entorno de desarrollo integrado (IDE), junto con **Jetpack Compose** para la creación de interfaces gráficas modernas en Android mediante Kotlin.

En el módulo de inteligencia artificial se utilizó **FastAPI** como framework para desarrollar el servicio encargado del procesamiento de texto y cálculo de compatibilidad.

En cuanto a herramientas de apoyo y desarrollo, se utilizaron:

- **Docker** y **Docker Compose** para contenerización y ejecución integrada de servicios.
- **MySQL Workbench** para administración y visualización de la base de datos.
- **Postman** para pruebas y validación de endpoints de la API REST.
- **GitHub** para control de versiones y trabajo colaborativo.
- **Visual Studio Code** como editor de código para frontend y backend.
- **Android Studio** para desarrollo y pruebas de la aplicación móvil.
- **Flyway** para el control de migraciones SQL versionadas.

b.5. Explicación del ambiente de pruebas y cómo réplica producción

El ambiente fue configurado utilizando Docker y Docker Compose, como ya se ha mencionado anteriormente, permitiendo ejecutar de forma conjunta los servicios principales del proyecto dentro de contenedores separados. Esta configuración contempla la ejecución del backend desarrollado en Spring Boot, la base de datos MySQL y el servicio de inteligencia artificial implementado en FastAPI. Gracias a esto, fue posible mantener una estructura organizada y consistente entre desarrollo y pruebas.

La aplicación móvil y la plataforma web consumen la misma API REST del backend, replicando el comportamiento esperado en producción. Tanto el frontend web como la aplicación Android realizan solicitudes HTTP reales hacia los endpoints del sistema, utilizando autenticación JWT y validando las reglas de negocio implementadas.

El ambiente de pruebas también considera el almacenamiento de archivos CV en el filesystem del servidor, manteniendo en MySQL únicamente la referencia del archivo, siguiendo la misma lógica definida para producción. Además, el backend se comunica mediante HTTP con el servicio FastAPI para realizar el cálculo de compatibilidad utilizando inteligencia artificial.

Durante el desarrollo se realizaron pruebas funcionales utilizando Postman para validar distintos endpoints de la API REST, incluyendo autenticación, registro de usuarios, creación de empresas y reclutadores, publicación de ofertas laborales, postulaciones y comunicación con el servicio de IA. Estas pruebas permitieron verificar el correcto funcionamiento de las rutas implementadas y la integración entre los distintos servicios del sistema.

Gracias a esta configuración, el equipo pudo trabajar sobre un entorno de pruebas estable y similar a producción, facilitando la detección temprana de errores, la validación de integraciones y la comprobación del funcionamiento general del sistema.

c) Evidencia técnica

c.1. Procedimiento de respaldo de base de datos

El procedimiento de respaldo consiste en generar copias de seguridad de la base de datos utilizando herramientas nativas de MySQL, principalmente mediante el comando `mysqldump`. Este proceso permite exportar toda la estructura y contenido de la base de datos en un archivo `.sql`, el cual puede ser restaurado posteriormente en caso de fallos, pérdida de información o migración del sistema.

El contenedor de base de datos se encuentra definido dentro del archivo `docker-compose.yml`, utilizando un volumen persistente (`mysql_data`) para asegurar que la información permanezca almacenada incluso si el contenedor es reiniciado o eliminado. Esto añade una segunda capa de protección frente a pérdida de datos.

Ejemplo de respaldo utilizado durante el desarrollo:

```
docker exec worksi-mysql mysqldump -u root -p worksi > respaldo_worksi.sql
```

Para restaurar un respaldo previamente generado, se utiliza el siguiente procedimiento:

```
docker exec -i worksi-mysql mysql -u root -p worksi < respaldo_worksi.sql
```

Además, el proyecto incorpora migraciones versionadas mediante Flyway, lo que permite mantener control sobre los cambios estructurales de la base de datos. Gracias a esto, cada modificación del esquema queda registrada mediante scripts SQL organizados por versiones, facilitando la recuperación y replicación consistente del entorno de datos.

Este enfoque permitió mantener estabilidad durante las pruebas del sistema y asegurar la continuidad del desarrollo colaborativo del equipo.

c.2. Configuración del servidor cloud/web

La infraestructura del sistema se compone de tres servicios principales definidos mediante Docker Compose:

- Base de datos MySQL 8
- Backend API REST desarrollado en Spring Boot
- Servicio de inteligencia artificial desarrollado en FastAPI

Cada servicio se ejecuta dentro de su propio contenedor, permitiendo independencia, escalabilidad y facilidad de despliegue. La comunicación entre servicios se realiza mediante red interna Docker utilizando nombres de servicio como host, evitando configuraciones manuales complejas.

El archivo `docker-compose.yml` define la orquestación completa del sistema, incluyendo dependencias entre servicios, variables de entorno, persistencia de datos y exposición de puertos para acceso externo.

La base de datos MySQL fue configurada utilizando persistencia mediante volúmenes Docker (`mysql_data`), asegurando la conservación de la información incluso ante reinicios o recreación de contenedores. Además, se implementó un `healthcheck` para validar automáticamente la disponibilidad del servicio antes de permitir el inicio del backend.

El backend Spring Boot fue configurado para ejecutarse en el puerto 8080, estableciendo conexión automática con MySQL y con el microservicio de inteligencia artificial. Para ello se utilizaron variables de entorno definidas dentro del contenedor, permitiendo desacoplar configuraciones sensibles del código fuente.

Por otra parte, el servicio IA basado en FastAPI fue configurado en el puerto 8000, encargado del procesamiento semántico entre currículums y ofertas laborales mediante algoritmos de matching.

La arquitectura definida permite replicar fácilmente el proyecto en distintos entornos, incluyendo:

- servidores locales,
- máquinas virtuales,
- infraestructura cloud,
- o despliegues futuros mediante plataformas como AWS, Azure o Google Cloud.

Esta configuración modular también facilita la escalabilidad del sistema, ya que cada componente puede evolucionar o desplegarse de manera independiente sin afectar el funcionamiento general de la plataforma.

c.3. Instalación de herramientas y librerías

Instalaciones del archivo POM del Backend:

```
<dependencies>

<!-- Spring Boot Web -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<!-- Spring Data JPA -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- MySQL Connector -->
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>

<!-- Spring Security -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<!-- Validaciones -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<!-- JWT -->
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.5</version>
</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
```

```

    <version>0.12.5</version>
    <scope>runtime</scope>
</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.5</version>
  <scope>runtime</scope>
</dependency>

<!-- Flyway -->
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-core</artifactId>
</dependency>

<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-mysql</artifactId>
</dependency>

<!-- Apache Tika -->
<dependency>
  <groupId>org.apache.tika</groupId>
  <artifactId>tika-core</artifactId>
  <version>2.9.2</version>
</dependency>

<dependency>
  <groupId>org.apache.tika</groupId>
  <artifactId>tika-parser-pdf-module</artifactId>
  <version>2.9.2</version>
</dependency>
</dependencies>

```

Instalaciones del FrontWeb, archivo package.json:

```

Plugins y configuración Android:
{
  "name": "frontweb",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {

```

```

    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "axios": "^1.16.0",
    "react": "^19.2.5",
    "react-dom": "^19.2.5",
    "react-router-dom": "^7.14.2"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "@types/react": "^19.2.14",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.1",
    "eslint": "^10.2.1",
    "eslint-plugin-react-hooks": "^7.1.1",
    "eslint-plugin-react-refresh": "^0.5.2",
    "globals": "^17.5.0",
    "vite": "^8.0.10"
  }
}

```

Instalaciones del FrontMovil, archivo build.gradle.kts:

```

plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
    id("org.jetbrains.kotlin.plugin.compose") version "2.0.21"
    id("kotlin-kapt")
}

android {
    namespace = "com.worksi.app"
    compileSdk = 35

    defaultConfig {
        applicationId = "com.worksi.app"
        minSdk = 26
        targetSdk = 35
        versionCode = 1
        versionName = "1.0.0"
    }

    buildFeatures {
        compose = true
    }
}

```

Dependencias principales:

```

dependencies {
    implementation("androidx.core:core-ktx:1.15.0")
}

```

```

implementation("androidx.activity:activity-compose:1.9.3")

implementation(platform("androidx.compose:compose-bom:2024.02.00"))
implementation("androidx.compose.ui:ui")
implementation("androidx.compose.material3:material3")
implementation("androidx.navigation:navigation-compose:2.8.5")

implementation("com.squareup.retrofit2:retrofit:2.11.0")
implementation("com.squareup.retrofit2:converter-moshi:2.11.0")
implementation("com.squareup.moshi:moshi-kotlin:1.15.1")

implementation("com.squareup.okhttp3:okhttp")
implementation("com.squareup.okhttp3:logging-interceptor")

implementation("androidx.security:security-crypto:1.1.0-alpha06")
implementation("io.coil-kt:coil-compose:2.7.0")
implementation("com.tom-roush:pdfbox-android:2.0.27.0")
}

```

Docker, archivo docker-compose.yml:

```

services:
  mysql:
    image: mysql:8.0
    container_name: worksi-mysql
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:-root}
      MYSQL_DATABASE: ${MYSQL_DATABASE:-worksi}
      MYSQL_USER: ${MYSQL_USER:-worksi}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD:-worksi}
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h 127.0.0.1 -uroot
      -p${MYSQL_ROOT_PASSWORD} --silent"]
      interval: 5s
      timeout: 5s
      retries: 30

  backend:
    build:
      context: ./backend
    container_name: worksi-backend
    depends_on:
      mysql:
        condition: service_healthy
      ai:
        condition: service_started
    environment:
      SPRING_DATASOURCE_URL:
jdbc:mysql://mysql:3306/${MYSQL_DATABASE:-worksi}?useSSL=false&allowPublicKeyR

```



```

etrieval=true&characterEncoding=utf8
  SPRING_DATASOURCE_USERNAME: ${MYSQL_USER:-worksi}
  SPRING_DATASOURCE_PASSWORD: ${MYSQL_PASSWORD:-worksi}
  WORKSI_CV_STORAGE_DIR: /data/cv
  WORKSI_SECURITY_PASSWORD_RECOVERY_MVP_SKIP_CODE_CHECK: "true"
  WORKSI_AI_BASE_URL: http://ai:8000
ports:
  - "8080:8080"
volumes:
  - cv_data:/data/cv

ai:
  build:
    context: ./ai-service
  container_name: worksi-ai
  ports:
    - "8000:8000"

volumes:
  mysql_data:
  cv_data:

```

Servicio de ia, archivo requirements:

```

fastapi==0.115.12
uvicorn[standard]==0.34.2
sentence-transformers==3.3.1
scikit-learn==1.5.2
spacy==3.8.2

```

c.4. Capturas de pantalla/Evidencia de funcionamiento

Las evidencias recopiladas permiten demostrar el correcto funcionamiento de:

- la arquitectura multicapa,
- el sistema de autenticación JWT,
- la conexión entre frontend y backend,
- la persistencia de datos en MySQL,
- el funcionamiento de contenedores Docker,
- y la integración del servicio de inteligencia artificial.

Entre las principales evidencias consideradas dentro del proyecto se encuentran:

- Levantamiento correcto de contenedores mediante Docker Compose.
- Funcionamiento del backend Spring Boot y servicio IA FastAPI.
- Respuesta satisfactoria del endpoint `/health`.
- Inicio de sesión exitoso mediante autenticación JWT.
- Consumo de endpoints REST.
- Registro y visualización de empresas, reclutadores y candidatos.

- Persistencia de información en base de datos MySQL.
- Visualización del frontend web desarrollado en React.
- Ejecución de la aplicación móvil Android desarrollada en Kotlin.
- Validación de endpoints protegidos según roles del sistema.
- Pruebas de recuperación de contraseña.
- Obtención de datos desde catálogos del sistema.
- Funcionamiento del sistema de publicaciones laborales.
- Integración operativa entre backend y microservicio de IA.

Las siguientes capturas muestran parte del funcionamiento real del sistema durante las pruebas realizadas en el entorno de desarrollo configurado para el proyecto.

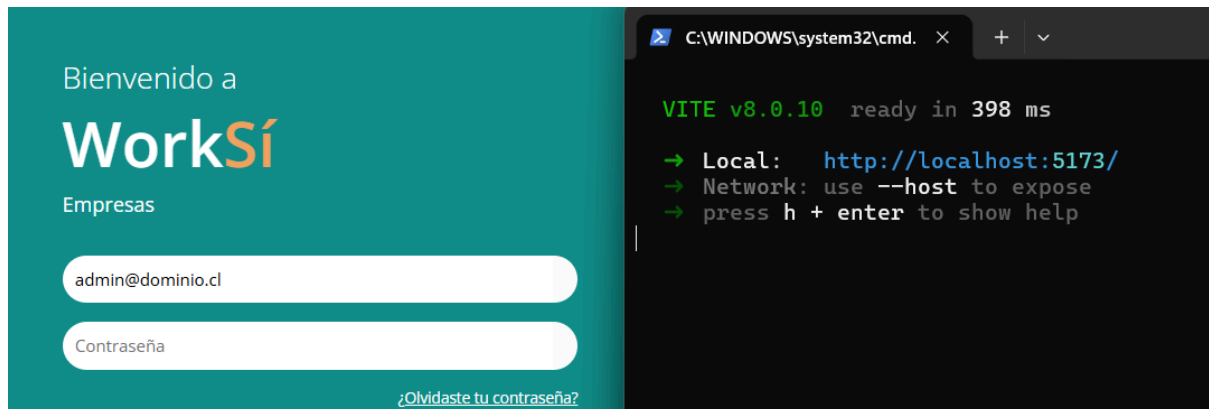
Demostración del Status de Docker con todos los servicios descritos previamente:

```
PS C:\Users\racue\Desktop> cd worksi-main
PS C:\Users\racue\Desktop\worksi-main> cd producto
PS C:\Users\racue\Desktop\worksi-main\producto> docker compose ps
```

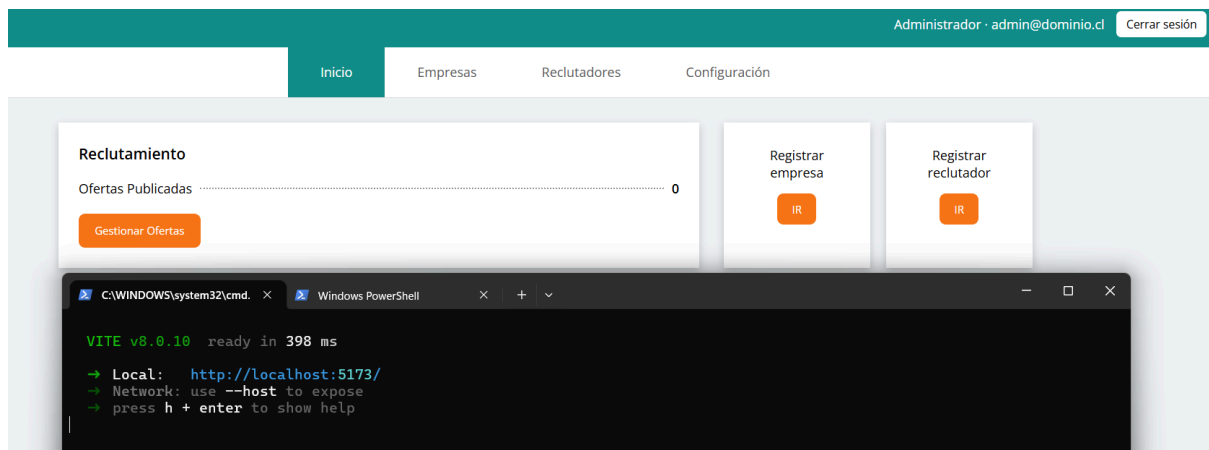
NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
worksi-ai	producto-ai	"uvicorn app.main:ap..."	ai	4 hours ago	Up 4 hours	0.0.0.0:8000
->8000/tcp, [::]:8000->8000/tcp						
worksi-backend	producto-backend	"java -jar /app/app..."	backend	4 hours ago	Up 4 hours	0.0.0.0:8080
->8080/tcp, [::]:8080->8080/tcp						
worksi-mysql	mysql:8.0	"docker-entrypoint.s..."	mysql	4 hours ago	Up 4 hours (healthy)	0.0.0.0:3306
->3306/tcp, [::]:3306->3306/tcp						

```
PS C:\Users\racue\Desktop\worksi-main\producto> |
```

Levantamiento de la web:



Admin:



Registro por parte del admin de un reclutador:

Registrar reclutadores

Nombre

Apellido materno

Correo de acceso

```
Type 'help;' or '\h' for help. Type '\c' to clear the current buffer.

mysql> USE worksi
Reading table information for completion of table and column names
You can turn off this feature to get a quicker start with MySQL.

Database changed
mysql> SELECT id, email, role
-> FROM users
-> WHERE role = 'RECRUITER';

+----+-----+-----+
| id | email           | role   |
+----+-----+-----+
|  2 | daily@ricardos.cl | RECRUITER |
+----+-----+-----+
```

Levantamiento del Front Movil:

```
1  pluginManagement {
2      repositories {
3          google {
4              content {
5                  includeGroupByRegex( groupRegex = "com\\.android.*")
6                  includeGroupByRegex( groupRegex = "com\\.google.*")
7                  includeGroupByRegex( groupRegex = "androidx.*")
8              }
9          }
10         mavenCentral()
11         gradlePluginPortal()
12     }
13 }
14 plugins {
15     id("org.gradle.toolchains.foojay-resolver-convention") version "1.0.0"
16 }
17 dependencyResolutionManagement {
18     repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
19     repositories {
20         google()
21         mavenCentral()
22     }
23 }
24
25 rootProject.name = "works1"
26 include( ...projectPaths = ":app")
```

5:51

Bienvenido a
WorkSÍ

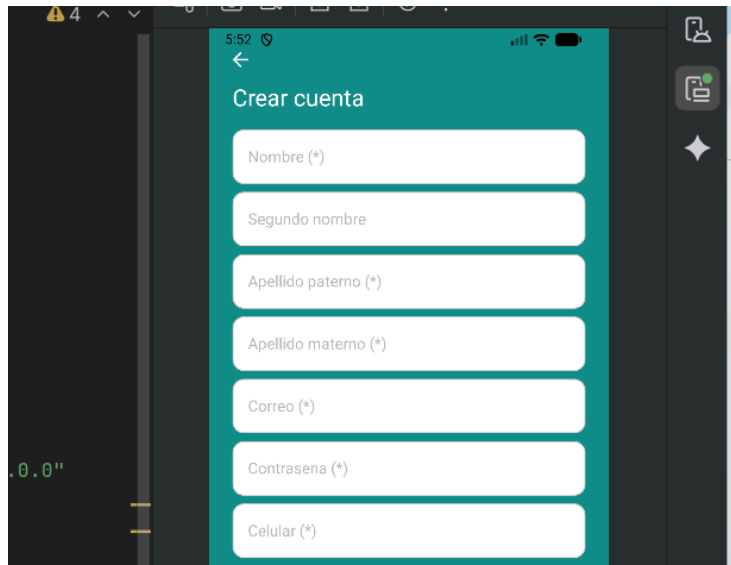
Tu nuevo trabajo a solo un **click**.

o si aún no eres parte del equipo

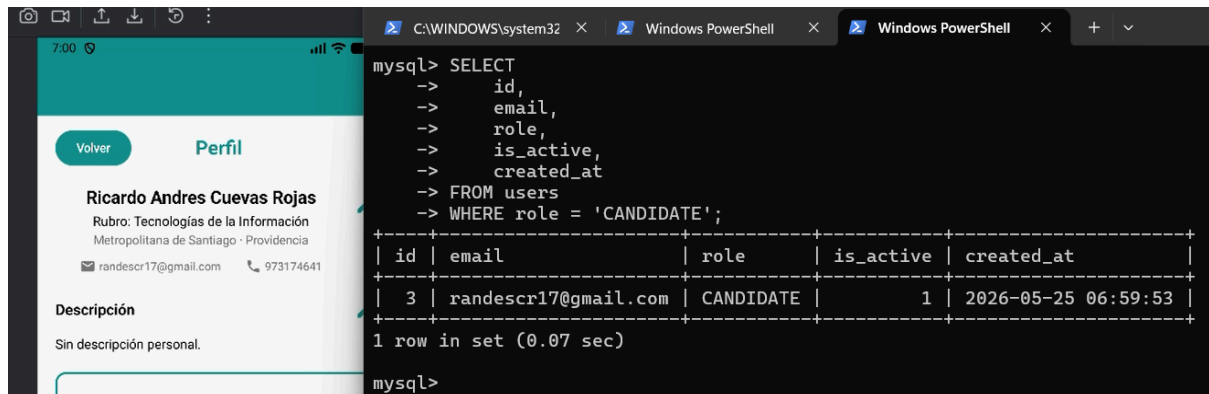
Iniciar Sesión

Crea tu cuenta

Creación de cuenta por parte del Postulante:



Datos de la base de datos del perfil postulante en Front Movil:



```
mysql> SELECT
-> id,
-> email,
-> role,
-> is_active,
-> created_at
-> FROM users
-> WHERE role = 'CANDIDATE';
+-----+-----+-----+-----+-----+
| id | email | role | is_active | created_at |
+-----+-----+-----+-----+-----+
| 3 | randescr17@gmail.com | CANDIDATE | 1 | 2026-05-25 06:59:53 |
+-----+-----+-----+-----+-----+
1 row in set (0.07 sec)

mysql>
```

d) Desarrollo del software

d.1. Avance funcional del sistema/Arquitectura implementada/Funcionalidades implementadas

Hasta la semana correspondiente al Sprint 7, el proyecto WorkSí presenta un avance funcional significativo tanto a nivel backend como frontend, integrando múltiples tecnologías y módulos operativos dentro de una arquitectura moderna basada en servicios desacoplados.

Durante el desarrollo del proyecto se logró implementar exitosamente la estructura principal de la plataforma, incluyendo autenticación, gestión de usuarios, administración de empresas, publicaciones laborales, perfiles de candidatos y conexión con un servicio de inteligencia artificial para el análisis de compatibilidad laboral.

En el backend desarrollado con Spring Boot se implementaron:

- endpoints REST funcionales,
- autenticación mediante JWT,
- control de acceso por roles (Administrador, Reclutador y Candidato),
- recuperación de contraseña,
- manejo de perfiles,
- administración de empresas,
- publicación de ofertas laborales,
- consumo de microservicios IA,
- y persistencia completa en MySQL.

A nivel de base de datos, se implementaron migraciones automáticas mediante Flyway, permitiendo mantener versionado y controlado el esquema relacional del sistema. En cuanto al **frontend web desarrollado con React**, actualmente se encuentran operativas funcionalidades asociadas a:

- autenticación de usuarios,
- panel administrativo,
- creación de empresas,
- creación de reclutadores,
- visualización de publicaciones laborales,
- y consumo de endpoints protegidos mediante JWT.

Por otra parte, la aplicación móvil Android desarrollada en Kotlin y Jetpack Compose cuenta con avances funcionales relacionados a:

- registro de candidatos,
- inicio de sesión,
- visualización del feed laboral,
- visualización de perfiles,
- consumo de API REST,
- almacenamiento seguro de tokens,
- y conexión con el backend principal.

Adicionalmente, se integró un microservicio IA desarrollado en FastAPI encargado de realizar procesos de matching entre currículums y ofertas laborales, permitiendo entregar puntajes de compatibilidad y explicaciones resumidas para mejorar la experiencia del candidato. Finalmente, durante las pruebas realizadas en Sprint 7, se logró ejecutar correctamente el sistema utilizando Docker Compose, permitiendo levantar de forma integrada:

- MySQL,
- Backend Spring Boot,
- Servicio IA FastAPI,
- Frontend web,
- y aplicación móvil Android,

validando así la comunicación completa entre los distintos componentes del sistema.

d.2. Buenas prácticas de programación

Durante el desarrollo del proyecto WorkSí se aplicaron diversas buenas prácticas de programación orientadas a mantener un código ordenado, escalable y mantenible.

En el backend desarrollado con Spring Boot se implementó una arquitectura por capas, separando responsabilidades entre:

- controllers,
- services,
- repositories,
- DTOs,
- configuraciones,
- y entidades.

Además, se utilizaron DTOs para desacoplar la comunicación de la API respecto de las entidades internas del sistema, mejorando la seguridad y organización del código.

En materia de seguridad, se implementó autenticación basada en JWT junto con control de acceso por roles, restringiendo funcionalidades según el tipo de usuario autenticado.

También se incorporaron validaciones mediante anotaciones como `@Valid` y `@NotBlank`, manejo centralizado de configuración mediante variables de entorno y propiedades externas, así como migraciones versionadas de base de datos utilizando Flyway.

Por otra parte, el proyecto fue modularizado utilizando servicios desacoplados, incluyendo un microservicio IA independiente desarrollado en FastAPI para el procesamiento de matching laboral.

Finalmente, se utilizó control de versiones mediante Git y GitHub, permitiendo trabajo colaborativo entre los integrantes del equipo y seguimiento continuo de los avances del proyecto.

d.3. Evidencia del código y funcionamiento

Como evidencia del desarrollo realizado hasta el Sprint 7, el proyecto WorkSí cuenta con implementación funcional en sus componentes principales: backend, frontend web, aplicación móvil, base de datos y servicio de inteligencia artificial. El código desarrollado demuestra una arquitectura organizada por módulos, separación de responsabilidades, consumo real de API REST, autenticación mediante JWT, validaciones funcionales y comunicación entre servicios.

La estructura del proyecto se encuentra organizada en distintos módulos dentro de la carpeta producto, separando el backend Spring Boot, el frontend web React/Vite, la aplicación móvil Android/Kotlin y el servicio de inteligencia artificial FastAPI. Esta organización permite trabajar de forma independiente cada componente del sistema, manteniendo una arquitectura clara y coherente con el diseño técnico del proyecto.

```
producto/
├── backend/
├── frontWeb/
├── frontmovil/
├── ai-service/
└── docker-compose.yml
```

Esta estructura evidencia que WorkSí fue desarrollado como una solución integrada, pero separada por responsabilidades, facilitando el mantenimiento, la escalabilidad y las pruebas de cada módulo.

Evidencia del backend Spring Boot

En el backend se implementó una arquitectura por capas, separando controladores, servicios, repositorios, DTOs, entidades, configuración, seguridad y validaciones. Esta estructura permite mantener un código más ordenado y facilita la evolución del sistema.

```
backend/src/main/java/cl/duoc/worksi/
├── client/
├── config/
├── controller/
├── dto/
├── entity/
├── exception/
├── repository/
├── security/
├── service/
├── util/
└── validation/
```

El backend expone endpoints REST para los principales módulos del sistema, incluyendo autenticación, administración, candidatos, empresas, catálogos y recuperación de contraseña. Un ejemplo de esto es el controlador de autenticación, encargado de recibir credenciales y delegar la lógica al servicio correspondiente:

```
@RestController
public class AuthController {
    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    @PostMapping("/api/v1/auth/login")
    public ResponseEntity<?> login(@Valid @RequestBody LoginRequest request) {
        return authService.login(request.getEmail(), request.getPassword());
    }
}
```


Este fragmento evidencia la implementación real del endpoint:

POST /api/v1/auth/login

El endpoint permite autenticar usuarios con rol ADMIN, RECRUITER o CANDIDATE, retornando un token JWT para acceder a rutas protegidas.

Evidencia de seguridad JWT y control de roles

El proyecto implementa seguridad mediante Spring Security y JWT. Las rutas están protegidas según el rol del usuario autenticado, separando claramente las funcionalidades de administrador, reclutador y candidato.

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http.csrf(csrf -> csrf.disable())
        .sessionManagement(s ->
            s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(
            auth ->
                auth.requestMatchers("/health").permitAll()
                    .requestMatchers(HttpMethod.GET, "/api/v1/catalogs/**").permitAll()
                    .requestMatchers(HttpMethod.POST, "/api/v1/auth/login").permitAll()
                    .requestMatchers(HttpMethod.POST,
                        "/api/v1/auth/register/candidate").permitAll()
                    .requestMatchers("/api/v1/auth/password-recovery/**").permitAll()
                    .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
                    .requestMatchers("/api/v1/company/**").hasRole("RECRUITER")
                    .requestMatchers("/api/v1/candidate/**").hasRole("CANDIDATE")
                    .anyRequest().permitAll()
                .addFilterBefore(jwtAuthenticationFilter,
                    UsernamePasswordAuthenticationFilter.class);

    return http.build();
}
```

Este código evidencia que el sistema trabaja de forma stateless, es decir, no mantiene sesiones en servidor. Cada solicitud protegida debe incluir un token JWT válido en el header Authorization.

Además, se configuró el uso de BCrypt para proteger las contraseñas almacenadas:

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Esto permite que las contraseñas no se guarden en texto plano en la base de datos, sino como hash seguro.

Evidencia del módulo administrador

El módulo administrador permite registrar empresas, listar empresas y crear reclutadores asociados a una empresa. Estas funciones están protegidas por rol ADMIN.

```
@RestController
@RequestMapping("/api/v1/admin")
public class AdminController {
    private final AdminCompanyService adminCompanyService;
    private final AdminRecruiterService adminRecruiterService;

    @PostMapping(value = "/companies", consumes =
MediaType.MULTIPART_FORM_DATA_VALUE)
    public ResponseEntity<?> createCompany(
        @RequestPart("data") String data,
        @RequestPart(value = "image", required = false) MultipartFile image) {
        return adminCompanyService.createCompany(data, image);
    }

    @GetMapping("/companies")
    public ResponseEntity<PageResponse<AdminCompanyListItem>> listCompanies(
        @RequestParam(name = "page", defaultValue = "1") int page,
        @RequestParam(name = "size", defaultValue = "20") int size,
        @RequestParam(name = "sort", defaultValue = "created_at,desc") String sort) {
        return adminCompanyService.listCompanies(page, size, sort);
    }

    @PostMapping(value = "/recruiters", consumes =
MediaType.APPLICATION_JSON_VALUE)
    public ResponseEntity<?> createRecruiter(@RequestBody AdminRecruiterRequest
body) {
        return adminRecruiterService.createRecruiter(body);
    }
}
```

Este fragmento evidencia que el administrador puede:

- POST /api/v1/admin/companies
- GET /api/v1/admin/companies
- POST /api/v1/admin/recruiters

Evidencia del módulo candidato

El flujo de candidatos se encuentra implementado mediante endpoints protegidos bajo el prefijo:

/api/v1/candidate

El controlador de ofertas para candidatos permite listar ofertas, ver detalles, cargar imágenes y registrar swipes:

```
@RestController
@RequestMapping("/api/v1/candidate")
public class CandidateJobsController {
    private final CandidateJobFeedService candidateJobFeedService;
    private final CandidateSwipeService candidateSwipeService;
    private final CompanyJobService companyJobService;

    @GetMapping("/jobs/feed")
    public ResponseEntity<PageResponse<CandidateJobFeedItemResponse>> feed(
        @AuthenticationPrincipal UserPrincipal principal,
        @RequestParam(name = "page", defaultValue = "1") int page,
        @RequestParam(name = "size", defaultValue = "20") int size) {
        return candidateJobFeedService.feed(principal.getUser().getId(), page, size);
    }

    @GetMapping("/jobs/{job_id}")
    public ResponseEntity<?> jobDetail(
        @AuthenticationPrincipal UserPrincipal principal,
        @PathVariable("job_id") long jobId) {
        return candidateJobFeedService.detail(principal.getUser().getId(), jobId);
    }

    @PostMapping("/swipes")
    public ResponseEntity<Void> swipe(
        @AuthenticationPrincipal UserPrincipal principal,
        @RequestBody CandidateSwipeRequest body) {
        candidateSwipeService.record(principal.getUser().getId(), body);
        return ResponseEntity.noContent().build();
    }
}
```

Este código evidencia funcionalidades importantes del candidato:

- GET /api/v1/candidate/jobs/feed
- GET /api/v1/candidate/jobs/{job_id}
- POST /api/v1/candidate/swipes

El request para registrar un swipe se implementó como un record, usando el identificador de la oferta y la acción realizada:

```
public record CandidateSwipeRequest(  
    @JsonProperty("job_id") long jobId,  
    SwipeAction action  
) {}
```

Esto permite diferenciar acciones como postular o descartar una oferta laboral dentro del flujo móvil.

Evidencia del perfil de candidato

El sistema permite consultar y actualizar el perfil del candidato autenticado:

```
@RestController  
@RequestMapping("/api/v1/candidate")  
public class CandidateProfileController {  
    private final CandidateProfileService candidateProfileService;  
  
    @GetMapping("/profile")  
    public ResponseEntity<?> getProfile(@AuthenticationPrincipal UserPrincipal principal) {  
        return candidateProfileService.getProfile(principal.getUser().getId());  
    }  
  
    @PatchMapping("/profile")  
    public ResponseEntity<?> patchProfile(  
        @AuthenticationPrincipal UserPrincipal principal,  
        @RequestBody CandidateProfilePatchRequest body) {  
        return candidateProfileService.patchProfile(principal.getUser().getId(), body);  
    }  
}
```

Evidencia del registro multipart de candidato

El registro de candidato se implementó mediante un único endpoint multipart, alineado con el flujo definido para la aplicación móvil.

```
@RestController  
public class CandidateRegistrationController {  
    private final CandidateRegistrationService candidateRegistrationService;  
  
    @PostMapping(  
        value = "/api/v1/auth/register/candidate",
```

```

        consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
public ResponseEntity<?> register(
    @RequestPart("data") String data,
    @RequestPart("file") MultipartFile file) {
    return candidateRegistrationService.register(data, file);
}
}

```

Este endpoint recibe:

data = JSON con datos del candidato

file = CV en PDF

El sistema no crea cuentas incompletas antes del consentimiento, sino que realiza el alta final cuando el candidato acepta el uso de datos y adjunta su CV.

Evidencia del módulo empresa y reclutador

El módulo de empresa/reclutador se encuentra bajo el prefijo:

/api/v1/company

El controlador permite obtener perfil de empresa, manejar imágenes, crear ofertas, listar ofertas y consultar detalle de publicaciones:

```

@RestController
@RequestMapping("/api/v1/company")
public class CompanyController {
    private final CompanyJobService companyJobService;

    @GetMapping("/profile")
    public ResponseEntity<?> getProfile(@AuthenticationPrincipal UserPrincipal principal) {
        return companyJobService.getRecruiterCompanyProfile(principal.getUser().getId());
    }

    @PostMapping(value = "/jobs", consumes =
        MediaType.MULTIPART_FORM_DATA_VALUE)
    public ResponseEntity<?> createJob(
        @AuthenticationPrincipal UserPrincipal principal,
        @RequestPart("data") String data,
        @RequestPart(value = "image", required = false) MultipartFile image) {
        return companyJobService.createJob(principal.getUser().getId(), data, image);
    }

    @GetMapping("/jobs")
    public ResponseEntity<?> listJobs(
        @AuthenticationPrincipal UserPrincipal principal,

```

```

    @RequestParam(name = "page", defaultValue = "1") int page,
    @RequestParam(name = "size", defaultValue = "20") int size,
    @RequestParam(name = "sort", defaultValue = "created_at,desc") String sort) {
    return companyJobService.listMyJobs(principal.getUser().getId(), page, size, sort);
}

@GetMapping("/jobs/{job_id}")
public ResponseEntity<?> getJob(
    @AuthenticationPrincipal UserPrincipal principal,
    @PathVariable("job_id") long jobId) {
    return companyJobService.getMyJob(principal.getUser().getId(), jobId);
}

```

Esto evidencia la implementación de los endpoints:

- GET /api/v1/company/profile
- POST /api/v1/company/jobs
- GET /api/v1/company/jobs
- GET /api/v1/company/jobs/{job_id}

El reclutador puede crear ofertas con imagen opcional, listar sus ofertas publicadas y visualizar el detalle de cada una.

Evidencia de integración con IA

El backend se comunica con el servicio de inteligencia artificial mediante un cliente HTTP específico:

```

@Service
public class AiMatchClient {
    private final RestClient worksiAiRestClient;
    private final WorksAiProperties props;

    public Optional<MatchApiResponse> match(String cvText, String jobText) {
        if (!props.isEnabled()) {
            return Optional.empty();
        }
        try {
            MatchApiResponse body =
                worksiAiRestClient
                    .post()
                    .uri("/match")
                    .contentType(MediaType.APPLICATION_JSON)
                    .body(new MatchApiRequest(cvText, jobText))
                    .retrieve()
                    .body(MatchApiResponse.class);

```

```

    if (body == null) {
        return Optional.empty();
    }

    return Optional.of(body);
} catch (RestClientException ex) {
    return Optional.empty();
}

```

Este código evidencia la comunicación entre Spring Boot y FastAPI mediante POST /match.

El request enviado al servicio IA contiene el texto del CV y el texto semántico de la oferta:

```

public class MatchApiRequest {
    @JsonProperty("cv_text")
    private final String cvText;

    @JsonProperty("job_text")
    private final String jobText;
}

```

La respuesta del servicio IA incluye el puntaje de compatibilidad y una explicación:

```

public class MatchApiResponse {
    private double score;
    private String explanation;
}

```

Además, el cliente HTTP hacia FastAPI se configura mediante `RestClient`, incluyendo URL base y tiempos de espera:

```

@Configuration
@EnableConfigurationProperties(WorksiAiProperties.class)
public class WorksiAiClientConfiguration {

    @Bean
    public RestClient worksiAiRestClient(WorksiAiProperties props) {
        SimpleClientHttpRequestFactory rf = new SimpleClientHttpRequestFactory();
        rf.setConnectTimeout(props.getConnectTimeoutMs());
        rf.setReadTimeout(props.getReadTimeoutMs());

        String base = props.getBaseUrl().trim();
        if (base.endsWith("/")) {
            base = base.substring(0, base.length() - 1);
        }
    }
}

```

```
    return RestClient.builder().baseUrl(base).requestFactory(rf).build();
  }
}
```

Esto evidencia una integración real entre backend e inteligencia artificial, desacoplando el cálculo del matching del backend principal.

Evidencia de catálogos públicos

El sistema cuenta con endpoints públicos para cargar regiones, comunas, sectores y skills. Estos datos son utilizados tanto por la aplicación web como por la aplicación móvil.

```
@RestController
public class CatalogController {
    private final CatalogService catalogService;

    @GetMapping("/api/v1/catalogs/regions")
    public CatalogListResponse listRegions() {
        return catalogService.listRegions();
    }

    @GetMapping("/api/v1/catalogs/regions/{region_id}/communes")
    public CatalogListResponse listCommunes(@PathVariable("region_id") long regionId) {
        return catalogService.listCommunesByRegion(regionId);
    }

    @GetMapping("/api/v1/catalogs/sectors")
    public CatalogListResponse listSectors() {
        return catalogService.listSectors();
    }

    @GetMapping("/api/v1/catalogs/sectors/{sector_id}/skills")
    public ResponseEntity<CatalogListResponse> listSkills(@PathVariable("sector_id") long sectorId) {
        Optional<CatalogListResponse> body = catalogService.listSkillsBySector(sectorId);
        return body.map(ResponseEntity::ok)
            .orElseGet(() -> ResponseEntity.status(HttpStatus.NOT_FOUND).build());
    }
}
```

Esto evidencia que la información de catálogos se consume desde base de datos mediante API REST, en lugar de estar fija en los frontends.

Evidencia del frontend web React

En el frontend web se implementó una navegación diferenciada según el rol autenticado. El componente Navbar lee la sesión desde `localStorage`, muestra opciones según el rol y permite cerrar sesión eliminando el token.

```
function readUser() {
  try {
    return JSON.parse(localStorage.getItem("user") || "{}");
  } catch {
    return {};
  }
}

function Navbar() {
  const navigate = useNavigate();
  const location = useLocation();
  const user = readUser();
  const role = user.role;

  const handleLogout = () => {
    localStorage.removeItem("token");
    localStorage.removeItem("user");
    navigate("/");
  };
}
```

Además, el sistema muestra una navegación distinta para reclutadores:

```
if (isRecruiterShell && role === "RECRUITER") {
  return (
    <div className="navbar-container">
      <div className="navbar-top">
        <h2 className="logo">
          Work<span>Sí</span>
        </h2>
        <div className="user-section">
          <span>
            {roleLabel} · {recruiterFullName(user) || user.email || ""}
          </span>
          <button type="button" onClick={handleLogout}>
            Cerrar sesión
          </button>
        </div>
      </div>
      <div className="navbar-menu">
        <div onClick={() => navigate("/recruiter/home")}>
```

```
      Inicio
    </div>
    <div onClick={() => navigate("/recruiter/reclutamiento")}>
      Reclutamiento
    </div>
  </div>
</div>
```

Esto evidencia que el frontend web adapta su navegación al tipo de usuario autenticado.

Evidencia de rutas protegidas en React

El componente `PrivateRoute` evita el acceso a rutas protegidas si no existe token o usuario válido. Además, redirige según el rol del usuario:

```
export function PrivateRoute({ children, roles }) {
  const token = localStorage.getItem("token");
  const user = readUser();
```

```
  if (!token || !user) {
    return <Navigate to="/" replace />;
  }

  const role = normalizeRole(user.role);

  if (!role) {
    return <Navigate to="/" replace />;
  }

  if (roles && roles.length > 0 && !roles.includes(role)) {
    if (role === "RECRUITER") {
      return <Navigate to="/recruiter/home" replace />;
    }
    if (role === "ADMIN") {
      return <Navigate to="/home" replace />;
    }
    return <Navigate to="/" replace />;
  }

  return children;
```

Esto demuestra que el frontend web no solo depende del backend para seguridad, sino que también aplica control de acceso en la interfaz, mejorando la experiencia del usuario.

Evidencia de login web

La pantalla de login del frontend web consume el servicio de autenticación y almacena el token y usuario retornados por el backend:

```
const data = await login(email, password);

const { token, user } = deriveSessionFromLoginBody(data);

localStorage.setItem("token", token);
localStorage.setItem("user", JSON.stringify(user));

const r = user.role;
if (r === "ADMIN") {
  navigate("/home");
} else if (r === "RECRUITER") {
  navigate("/recruiter/home");
} else {
  navigate("/home");
}
```

También se implementó control de bloqueo por intentos fallidos, derivando al flujo de recuperación de contraseña cuando corresponde:

```
const blocked = parseLoginBlockedDerivation(err);
if (blocked) {
  navigate("/recovery/locked", {
    state: {
      email: email.trim(),
      lockReason: blocked.lockReason,
    },
  });
  return;
}
```

Esto evidencia que el login web está conectado al backend y responde a reglas de negocio reales, incluyendo autenticación por roles y recuperación MVP.

Evidencia de creación de ofertas en frontend web

El formulario de creación de ofertas implementa validaciones locales antes de enviar la información al backend. Se validan campos obligatorios, región, comuna, sueldo, experiencia, modalidad, carga horaria, skills e imagen.

```
const validate = () => {
  const e = emptyErrors();
  let ok = true;
```

```

const req = (key) => {
  if (!String(form[key] ?? "").trim()) {
    e[key] = "Completa este campo";
    ok = false;
  }
};

req("company_commercial_name");
req("title");
req("description");
req("commune_id");
req("salary_offered");
req("years_experience_required");
req("modality");
req("workload");

if (!regionId) {
  e.region_id = "Completa este campo";
  ok = false;
}
if (selectedSkillIds.size < 3 || selectedSkillIds.size > 8) {
  e.skills = "Elija entre 3 y 8 skills";
  ok = false;
}
if (jobImageFile) {
  const t = jobImageFile.type || "";
  if (t !== "image/png" && t !== "image/jpeg") {
    e.image_file = "Solo PNG o JPEG";
    ok = false;
  }
}
setFieldErrors(e);
return ok;
}

```

Al confirmar, se construye el payload y se envía como oferta laboral:

```

const payload = {
  company_commercial_name: form.company_commercial_name.trim(),
  title: form.title.trim(),
  description: form.description.trim(),
  region_id: Number(regionId),
  commune_id: Number(form.commune_id),
  salary_offered: Number(form.salary_offered),
  years_experience_required: Number(form.years_experience_required),

```

```
modality: form.modality,  
workload: form.workload,  
skills_ids: Array.from(selectedSkillIds),  
};  
  
await createJob(payload, jobImageFile);
```

Este código evidencia el funcionamiento del módulo reclutador para publicar ofertas laborales desde el frontend web.

Evidencia de la aplicación móvil Android

La aplicación móvil implementa navegación mediante Jetpack Compose Navigation. El flujo incluye splash, bienvenida, login, registro, carga de CV, selección de skills, preferencias, consentimiento, recuperación de contraseña, home, perfil y detalle de oferta.

```
sealed class Screen(val route: String) {  
    object Splash : Screen("splash")  
    object Welcome : Screen("welcome")  
    object Login : Screen("login")  
    object Session : Screen("session")  
    object Profile : Screen("profile")  
    object RegisterPersonal : Screen("register_personal")  
    object RegisterCv : Screen("register_cv")  
    object RegisterSkills : Screen("register_skills")  
    object RegisterPreferences : Screen("register_preferences")  
    object RegisterConsent : Screen("register_consent")  
    object RecoveryLocked : Screen("recovery_locked")  
    object RecoveryEmail : Screen("recovery_email")  
    object RecoveryCode : Screen("recovery_code")  
    object RecoveryNewPassword : Screen("recovery_new_password")  
    object RecoverySuccess : Screen("recovery_success")  
}
```

Evidencia del registro móvil con estado local

El registro del candidato en Android utiliza un `ViewModel` para mantener los datos del usuario a través de varias pantallas antes del envío final al backend.

```
data class RegisterDraft(  
    val firstName: String = "",  
    val middleName: String = "",  
    val lastNamePaternal: String = "",  
    val lastNameMaternal: String = "",  
    val email: String = "",
```

```

val password: String = "",
val phone: String = "",
val rut: String = "",
val documentNumber: String = "",
val street: String = "",
val regionId: Long? = null,
val comunelId: Long? = null,
val sectorId: Long? = null,
val skillIds: Set<Long> = emptySet(),
val profileSummary: String = "",
val salaryMin: String = "",
val salaryMax: String = "",
val modalities: Set<String> = emptySet(),
val workloads: Set<String> = emptySet(),
val cvUri: String? = null,
val cvDisplayName: String? = null,
val consentAccepted: Boolean = false

```

Esto evidencia que los datos del candidato se mantienen en memoria durante el flujo móvil, sin crear una cuenta hasta aceptar el consentimiento.

Evidencia del envío multipart desde Android

Al finalizar el registro, la app construye un JSON con los datos del candidato y lo envía junto con el CV en PDF mediante multipart:

```

val payload =
    CandidateRegisterPayload(
        email = d.email.trim(),
        password = d.password,
        firstName = d.firstName.trim(),
        middleName = mid,
        lastNamePaternal = d.lastNamePaternal.trim(),
        lastNameMaternal = d.lastNameMaternal.trim(),
        phone = d.phone.trim(),
        rut = d.rut.trim(),
        documentNumber = d.documentNumber.trim(),
        street = street,
        regionId = d.regionId!!,
        comunelId = d.comunelId!!,
        consentGiven = true,
        sectorId = d.sectorId!!,
        profileSummary = summary,
        salaryExpectedMin = salaryMinOut,
        salaryExpectedMax = salaryMaxOut,
        preferredModalities = d.modalities.toList(),
    )

```

```
preferredWorkloads = d.workloads.toList(),
skillsIds = d.skillsIds.toList()
)
```

Luego convierte el payload a JSON y adjunta el CV como archivo PDF, si el registro es exitoso, guarda el token JWT. Esto demuestra que el flujo móvil realiza el alta oficial del candidato contra el backend y deja al usuario autenticado.

Evidencia de Retrofit y consumo de API móvil

La app móvil utiliza Retrofit para consumir los endpoints del backend:

```
interface AuthService {
    @POST("api/v1/auth/login")
    suspend fun login(@Body request: LoginRequest): Response<LoginResponse>

    @Multipart
    @POST("api/v1/auth/register/candidate")
    suspend fun registerCandidate(
        @Part("data") data: RequestBody,
        @Part file: MultipartBody.Part
    ): Response<CandidateRegisterResponseDto>

    @POST("api/v1/auth/password-recovery/request")
    suspend fun passwordRecoveryRequest(
        @Body body: PasswordRecoveryRequestBody
    ): Response<PasswordRecoveryRequestResponse>

    @POST("api/v1/auth/password-recovery/verify")
    suspend fun passwordRecoveryVerify(
        @Body body: PasswordRecoveryVerifyBody
    ): Response<PasswordRecoveryVerifyResponse>

    @POST("api/v1/auth/password-recovery/reset")
    suspend fun passwordRecoveryReset(
        @Body body: PasswordRecoveryResetBody
    ): Response<PasswordRecoveryResetResponse>
}
```

Evidencia de Docker Compose y funcionamiento integrado

El proyecto utiliza Docker Compose para levantar los servicios principales del sistema:

```
services:
  mysql:
    image: mysql:8.0
    container_name: worksi-mysql
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql

  backend:
    build:
      context: ./backend
    container_name: worksi-backend
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/works
      WORKSI_CV_STORAGE_DIR: /data/cv
      WORKSI_AI_BASE_URL: http://ai:8000
    ports:
      - "8080:8080"
    volumes:
      - cv_data:/data/cv

  ai:
    build:
      context: ./ai-service
    container_name: worksi-ai
    ports:
      - "8000:8000"

volumes:
  mysql_data:
  cv_data:
```

Esto permite ejecutar de forma conjunta:

- MySQL
- Spring Boot
- FastAPI

En conjunto, la evidencia de código y funcionamiento demuestra que WorkSí no se encuentra solo en etapa de diseño, sino que posee una implementación funcional avanzada hasta el Sprint 7. El sistema ya cuenta con autenticación, roles, rutas protegidas, registro de

candidatos, administración de empresas y reclutadores, publicación de ofertas, consumo desde frontend web, flujo móvil de candidato, almacenamiento seguro de token, integración con base de datos y comunicación con el servicio de inteligencia artificial.

Esto permite afirmar que la arquitectura central del proyecto se encuentra implementada y validada, quedando como próximos pasos completar la integración total del flujo de postulaciones, fortalecer las pruebas finales y consolidar la evidencia funcional para la entrega del MVP.

e) Plan y documentación de pruebas

e.1. Casos de prueba/Validaciones realizadas

Durante las pruebas del sistema WorkSí se utilizaron solicitudes HTTP mediante Postman para validar los principales endpoints del backend, junto con consultas directas a MySQL para comprobar la persistencia de los datos. Estas pruebas permitieron verificar el correcto funcionamiento de la autenticación, control de roles, catálogos, administración de empresas, reclutadores, ofertas laborales, recuperación de contraseña y comunicación con la aplicación web y móvil.

Caso de prueba	Endpoint / acción	Datos o condición probada	Resultado esperado	Resultado obtenido
Verificar disponibilidad del backend	GET /health	Backend ejecutándose en Docker	El sistema responde estado activo	Respuesta correcta con estado UP
Inicio de sesión válido	POST /api/v1/auth/login	Credenciales correctas de administrador o reclutador	Generación de token JWT y datos del usuario	Login exitoso y token recibido
Inicio de sesión inválido	POST /api/v1/auth/login	Credenciales incorrectas	Mensaje de error y rechazo de acceso	El sistema rechaza el login

Validación de rutas protegidas	GET /api/v1/admin/companies	Acceso con token ADMIN	Permitir acceso al listado de empresas	Acceso autorizado correctamente
Validación de permisos por rol	GET /api/v1/admin/companies	Acceso con token RECRUITER	Denegar acceso por rol incorrecto	El sistema bloquea el acceso
Consulta de catálogos	GET /api/v1/catalogs/regions	Solicitud pública sin token	Retornar listado de regiones	Catálogo cargado correctamente
Registro de reclutador	POST /api/v1/admin/recruiters	Datos válidos de reclutador y empresa asociada	Crear usuario reclutador	Reclutador registrado en base de datos
Consulta de perfil de empresa	GET /api/v1/company/profile	Token de reclutador válido	Mostrar datos de la empresa asociada	Perfil cargado correctamente
Creación de oferta laboral	POST /api/v1/company/jobs	Datos de oferta, skills e imagen opcional	Registrar oferta laboral	Oferta creada correctamente
Listado de ofertas del reclutador	GET /api/v1/company/jobs	Token de reclutador válido	Mostrar ofertas publicadas por la empresa	Listado obtenido correctamente
Feed de ofertas para candidato	GET /api/v1/candidate/jobs/feed	Token de candidato válido	Mostrar ofertas activas disponibles	Feed cargado desde backend

Recuperación de contraseña	POST /api/v1/auth/password-recovery/request	Correo registrado	Iniciar flujo de recuperación	Solicitud procesada correctamente
Verificación de recuperación	POST /api/v1/auth/password-recovery/verify	Código de recuperación MVP	Generar token de recuperación	Token generado correctamente
Cambio de contraseña	POST /api/v1/auth/password-recovery/reset	Nueva contraseña válida	Actualizar contraseña del usuario	Contraseña modificada correctamente

Además de las pruebas mediante Postman, se realizaron consultas directas en MySQL para comprobar que los datos generados por el sistema fueran almacenados correctamente.

Consulta para verificar usuarios reclutadores registrados:

- SELECT id, email, role
- FROM users
- WHERE role = 'RECRUITER';

Consulta para verificar usuarios candidatos registrados:

- SELECT id, email, role
- FROM users
- WHERE role = 'CANDIDATE';

Consulta para verificar ofertas laborales almacenadas y asociadas a una empresa:

- SELECT
- j.id,
- j.title,
- c.commercial_name AS company,
- j.salary_offered,
- j.modality,
- j.workload,
- j.created_at
- FROM jobs j
- JOIN companies c ON j.company_id = c.id;

Estas consultas permitieron comprobar que los usuarios creados desde el sistema se almacenan correctamente, que los roles son asignados de forma adecuada y que las ofertas laborales quedan persistidas en la base de datos junto con su empresa correspondiente.

En conjunto, los casos de prueba permitieron validar que el backend responde correctamente, que los tokens JWT se generan y se utilizan en rutas protegidas, que el control de acceso por roles funciona de acuerdo con el tipo de usuario, que los catálogos se obtienen desde la base de datos, que las empresas, reclutadores y ofertas laborales se almacenan correctamente, y que tanto la aplicación web como la aplicación móvil pueden comunicarse con la API REST. Además, se dejó preparada la integración con el servicio de inteligencia artificial para el cálculo de compatibilidad entre candidatos y ofertas laborales.

e.2. Resultados obtenidos/Evidencia de pruebas ejecutadas

A partir de las pruebas realizadas, se comprobó que los principales módulos del sistema WorkSí funcionan correctamente según el avance alcanzado hasta el Sprint 7.

El backend respondió de forma correcta mediante el endpoint GET /health, confirmando que el servicio principal se encontraba activo. También se validó el inicio de sesión con POST /api/v1/auth/login, obteniendo tokens JWT válidos para acceder a rutas protegidas.

Se verificó que el control de acceso por roles funciona adecuadamente, permitiendo al administrador acceder a funciones como la gestión de empresas y reclutadores, mientras que el reclutador pudo ingresar a las funcionalidades relacionadas con su empresa y la publicación de ofertas laborales.

Los catálogos del sistema, como regiones, comunas, rubros y skills, fueron consultados correctamente desde el backend, demostrando que los datos base pueden ser utilizados tanto por la aplicación web como por la aplicación móvil.

En el módulo administrativo, se comprobó el registro y almacenamiento de empresas y reclutadores en la base de datos MySQL. Asimismo, en el módulo del reclutador se validó la creación, listado y visualización de ofertas laborales, confirmando que la información se guarda y recupera correctamente.

Desde la aplicación web, se verificó el inicio de sesión, la navegación según rol, la protección de rutas y el acceso a las funciones principales de administración y reclutamiento. En la aplicación móvil, se comprobó el flujo de candidato, incluyendo navegación, registro, carga de CV, selección de skills, preferencias y consumo de la API mediante Retrofit.

Finalmente, mediante consultas SQL se confirmó la persistencia de usuarios, roles y ofertas laborales en MySQL. También se verificó que el servicio de inteligencia artificial se encuentra integrado en la arquitectura y preparado para el cálculo de compatibilidad entre candidatos y ofertas.

A continuación la evidencia en algunas imágenes de lo mencionado anteriormente:

Registrar reclutadores

Nombre

Apellido materno

Correo de acceso

```
Type 'help;' or '\h' for help. Type '\c' to clear th

mysql> USE worksi
Reading table information for completion of table ar
You can turn off this feature to get a quicker start

Database changed
mysql> SELECT id, email, role
-> FROM users
-> WHERE role = 'RECRUITER';

+----+-----+-----+
| id | email           | role   |
+----+-----+-----+
| 2  | daily@ricardos.cl | RECRUITER |
+----+-----+-----+
```

Chilexpress
 Correo corporativo: chilexpress@chilexpress.cl
 Daily zamudio molina
 daily@chilexpress.cl — Reclutador

Reclutamiento
 Ofertas publicadas
 Ir a ver

```
1 row in set (0.00 sec)

mysql> SELECT id, email, role
-> FROM users
-> WHERE role = 'CANDIDATE';
Empty set (0.00 sec)

mysql> SELECT
-> id,
-> email,
-> role,
-> is_active,
-> created_at
-> FROM users
-> WHERE role = 'RECRUITER';

+----+-----+-----+-----+-----+
| id | email           | role   | is_active | created_at |
+----+-----+-----+-----+-----+
| 2  | daily@ricardos.cl | RECRUITER | 1 | 2026-05-25 02:41:33 |
| 4  | daily@chilexpress.cl | RECRUITER | 1 | 2026-05-25 07:06:33 |
+----+-----+-----+-----+-----+

2 rows in set (0.05 sec)

mysql>
```

Perfil

Ricardo Andres Cuevas Rojas
 Rubro: Tecnologías de la Información
 Metropolitana de Santiago · Providencia
 randescr17@gmail.com 973174641

Descripción
 Sin descripción personal.

```
mysql> SELECT
-> id,
-> email,
-> role,
-> is_active,
-> created_at
-> FROM users
-> WHERE role = 'CANDIDATE';

+----+-----+-----+-----+-----+
| id | email           | role   | is_active | created_at |
+----+-----+-----+-----+-----+
| 3  | randescr17@gmail.com | CANDIDATE | 1 | 2026-05-25 06:59:53 |
+----+-----+-----+-----+-----+

1 row in set (0.07 sec)

mysql>
```

POST http://localhost:8080/api/v1/auth/login

Docs Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "email": "admin@dominio.cl",
3   "password": "Admin1234*"
4 }
```

Body Cookies Headers (14) Test Results 200 OK • 2.50 s • 796 B

{ } JSON Preview Visualize

```
1 {
2   "access_token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiIxIiwiaWZwIjoiJG1pbk8kb21pbm1vLmNsIiwicm9sZSI6IkFETU0IiwiaWF0IjoxNzc5Njk0NDA1LCJleHAiOjE3Nzk2OTgwMDV9.ywCZDdUzRORGTxb5_RzQm5lu04R0wKw6K4HA1KzHkYk",
3   "token_type": "Bearer",
4   "expires_in": 3600,
5   "user": {
6     "id": 1,
7     "role": "ADMIN",
8     "email": "admin@dominio.cl"
9   },
10  "lock_info": {
11    "is_locked": false,
12    "failed_attempts": 0,
13    "remaining_attempts": 4
14  }
15 }
```

GET http://localhost:8080/api/v1/catalogs/regions

Docs Params Authorization Headers (7) Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (14) Test Results 200 OK • 297 ms • 1.21 KB

{ } JSON Preview Visualize

```
1 {
2   "items": [
3     {
4       "id": 3,
5       "code": "CL-AN",
6       "name": "Antofagasta"
7     },
8     {
9       "id": 1,
10      "code": "CL-AP",
11      "name": "Arica y Parinacota"
12    }
13  ]
14 }
```

GET http://localhost:8080/api/v1/admin/companies

Auth Type: Bearer Token

Token:

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body: 200 OK • 2.12 s • 751 B

```
{
  "items": [
    {
      "company_id": 2,
      "commercial_name": "Chilexpress",
      "legal_name": "Curier",
      "rut": "96756430-3",
      "corporate_email": "chilexpress@chilexpress.cl"
    },
    {
      "company_id": 1,
      "commercial_name": "Ricarditos",
      "legal_name": "Venta",
      "rut": "20327548-K",
      "corporate_email": "randescr12@gmail.com"
    }
  ],
  "page": 1
}
```

POST http://localhost:8080/api/v1/admin/recruiters

Body: 201 Created • 2.53 s • 508 B

```
{
  "email": "reclutador2@empresa.cl",
  "password": "Reclutador123*",
  "first_name": "Pedro",
  "last_name_paternal": "Gonzalez",
  "last_name_maternal": "Rojas",
  "rut": "22222222-2",
  "phone": "123456789",
  "mobile": "987654321",
  "birth_date": "1994-01-01",
  "company_id": 1
}
```

Body: 201 Created • 2.53 s • 508 B

```
{
  "user_id": 5,
  "role": "RECRUITER",
  "email": "reclutador2@empresa.cl",
  "company_id": 1
}
```

GET http://localhost:8080/api/v1/company/jobs

Docs Params Authorization Headers (7) Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (14) Test Results 401 Unauthorized 66 ms 537 B

JSON Preview Pass the correct auth credentials

```
1 {
2   "error": {
3     "message": "Se requiere autenticacion",
4     "trace_id": "",
5     "code": "UNAUTHORIZED",
6     "details": []
7   }
8 }
```

GET http://localhost:8000/openapi.json

Docs Params Authorization Headers (7) Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (4) Test Results 200 OK

JSON Preview Visualize

```
1 {
2   "openapi": "3.1.0",
3   "info": {
4     "title": "FastAPI",
5     "version": "0.1.0"
6   },
7   "paths": {
8     "/health": {
9       "get": {
10        "summary": "Health",
11        "operationId": "health_health_get",
12        "responses": {
13          "200": {
14            "description": "Successful Response",
15            "content": {
16              "application/json": {
17                "schema": {}
18              }
19            }
20          }
21        }
22      }
23    }
24  }
```


6. CONCLUSIONES

6.1. Estado actual del proyecto

Actualmente, el proyecto WorkSí se encuentra en una etapa avanzada de desarrollo correspondiente al avance alcanzado hasta el Sprint 7. La base principal del sistema ya está implementada, incluyendo el backend, la base de datos, parte importante del frontend web, la aplicación móvil y la integración inicial con el servicio de inteligencia artificial.

El backend desarrollado en Spring Boot cuenta con endpoints funcionales para autenticación, recuperación de contraseña, administración de empresas, registro de reclutadores, gestión de perfil de empresa, creación y consulta de ofertas laborales, catálogos, registro de candidatos, perfil de candidato y feed de ofertas. Además, se implementó seguridad mediante JWT y control de acceso por roles, diferenciando las acciones permitidas para administradores, reclutadores y candidatos.

La base de datos MySQL se encuentra estructurada mediante migraciones con Flyway, permitiendo almacenar usuarios, roles, empresas, reclutadores, candidatos, CV, ofertas laborales, postulaciones, catálogos y preferencias. También se verificó la persistencia de datos mediante consultas SQL directas.

En el frontend web, desarrollado con React y Vite, ya se cuenta con login, navegación según rol, rutas protegidas, panel administrativo, registro de empresas, registro de reclutadores, módulo de reclutador, creación de ofertas, listado de ofertas y visualización del detalle de una oferta publicada. Esto permite que el administrador y el reclutador interactúen con funcionalidades reales conectadas al backend.

En la aplicación móvil Android, desarrollada con Kotlin y Jetpack Compose, se encuentra implementado el flujo principal del candidato, incluyendo navegación, login, registro por etapas, carga de CV en PDF, selección de skills, preferencias laborales, consentimiento de uso de datos, perfil, home y visualización de ofertas. La app consume la API mediante Retrofit y almacena el token de sesión de forma segura.

El servicio de inteligencia artificial con FastAPI se encuentra integrado dentro de la arquitectura del sistema y preparado para apoyar el cálculo de compatibilidad entre candidatos y ofertas laborales. El backend puede comunicarse con este servicio para enviar el texto del CV y la información de la oferta, obteniendo un puntaje de compatibilidad y una explicación del resultado.

En términos generales, WorkSí ya cuenta con una base funcional sólida. Las funcionalidades principales del MVP están implementadas o en proceso de integración final, mientras que los próximos pasos se orientan a completar pruebas, mejorar detalles visuales, reforzar validaciones, consolidar evidencias y finalizar los módulos pendientes relacionados con postulaciones, revisión de candidatos y resultados del matching.

6.2. Dificultades encontradas

Durante el desarrollo del proyecto, una de las principales dificultades fue lograr levantar correctamente el ambiente de trabajo con Docker y Docker Compose, especialmente por la configuración de los contenedores del backend, MySQL y el servicio de inteligencia artificial. Fue necesario ajustar variables de entorno, puertos, conexión entre servicios y verificar que cada contenedor respondiera correctamente.

Otra dificultad importante fue el modelado de la base de datos, ya que el sistema requiere relacionar distintos actores y entidades, como usuarios, candidatos, reclutadores, empresas, ofertas, postulaciones, CV, skills y preferencias. Definir estas relaciones de forma ordenada fue fundamental para evitar problemas posteriores en el desarrollo.

En general, la mayor complejidad no estuvo solo en programar la lógica de cada módulo, sino en integrar correctamente todas las partes del sistema para que trabajaran en conjunto: backend, base de datos, frontend web, aplicación móvil y servicio de IA.

6.3. Próximos pasos

Debido a que el proyecto WorkSí presenta un avance mayor al esperado para la etapa actual, se considera la posibilidad de incorporar mejoras adicionales durante 2 sprints nuevos. Estos sprints permitirían fortalecer el MVP, mejorar la experiencia de usuario y agregar funcionalidades complementarias que aporten mayor valor al sistema.

En primer lugar, se propone terminar de consolidar los flujos principales ya desarrollados, especialmente el proceso completo de postulación del candidato, la revisión de postulantes por parte del reclutador y la visualización del porcentaje de compatibilidad generado por la IA. Esto permitiría cerrar el ciclo principal del sistema: publicación de oferta, postulación, matching y revisión del candidato.

Gracias al esfuerzo del equipo, la funcionalidad más importante que ahora se podrá agregar al sistema (programada para nuevo Sprint 11) es la funcionalidad del **match real entre reclutador y candidato**. Se trata de una funcionalidad atractiva que se descartó inicialmente, pero ahora que se cuenta con 2 sprints adicionales podrá incluirse, permitiendo que el sistema no sólo ofrezca la visualización de porcentaje de matching entre candidatos y ofertas, si no también la posibilidad de establecer match entre reclutadores y candidatos, lo cuál indicaría que el reclutador sí o sí contactará al candidato para citarlo a fases posteriores del reclutamiento.

Otro próximo paso importante sería ampliar las pruebas del sistema, incorporando pruebas más completas de extremo a extremo para validar el funcionamiento integrado entre backend, base de datos, frontend web, aplicación móvil y servicio de IA. Además, se podrían documentar mejor los resultados obtenidos mediante capturas de pantalla, casos de prueba y evidencias técnicas.

Finalmente, estos sprints adicionales permitirían corregir detalles pendientes, optimizar el rendimiento general, mejorar validaciones y preparar una versión más completa y presentable del MVP para la entrega final del proyecto.

7. LECCIONES APRENDIDAS

Durante el desarrollo de WorkSí se aprendió que un proyecto de software no depende únicamente de programar funcionalidades, sino también de lograr que todos sus componentes trabajen correctamente en conjunto. La integración entre backend, base de datos, frontend web, aplicación móvil y servicio de inteligencia artificial fue clave para comprender la importancia de una arquitectura bien organizada.

Una de las principales lecciones fue la importancia de planificar correctamente la base de datos desde el inicio. Al existir distintos tipos de usuarios, empresas, reclutadores, candidatos, ofertas, postulaciones, skills y preferencias, fue necesario modelar las relaciones con cuidado para evitar problemas posteriores en el desarrollo.

También se aprendió la relevancia de trabajar con ambientes controlados mediante Docker. Aunque al comienzo levantar los contenedores fue una de las tareas más complejas, permitió entender mejor cómo se comunican los servicios, cómo se configuran variables de entorno y cómo se puede replicar un ambiente similar al de producción.

Además, este proyecto permitió aprender por primera vez a integrar inteligencia artificial dentro de una solución real. La conexión entre el backend y el servicio de IA ayudó a comprender cómo un sistema puede utilizar modelos externos para analizar información, generar un porcentaje de compatibilidad y apoyar la toma de decisiones dentro del proceso de reclutamiento.

Finalmente, el desarrollo del proyecto permitió reforzar la importancia de probar constantemente cada avance. El uso de Postman, consultas SQL, pruebas desde la web y pruebas desde la aplicación móvil ayudó a detectar errores a tiempo y confirmar que las funcionalidades realmente estaban conectadas y funcionando correctamente.